

Ponteiros

Computação Eletrônica

Juliano M. Iyoda
jmi@cin.ufpe.br

Ponteiros

- Cada pedaço da memória vem com um número de fábrica
- Estes números são os endereços da memória
 - Assim como nós, as “gavetas” da memória também têm endereços

Memória

000	001	002
003	004	005
006	007	008

Ponteiros

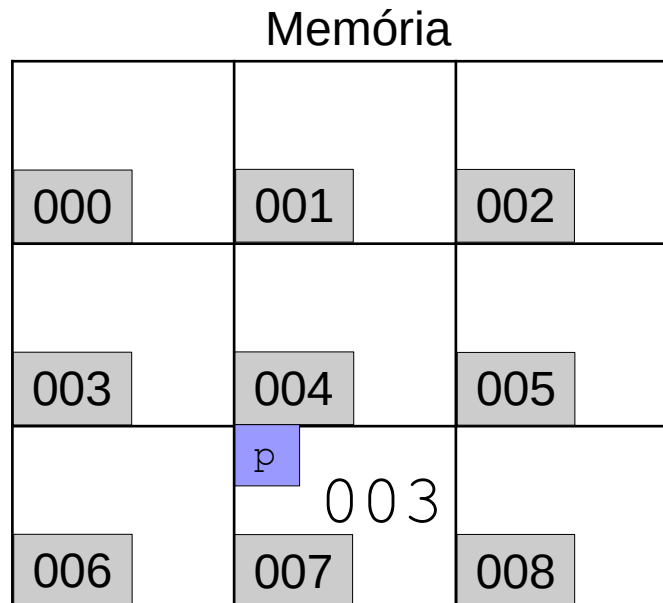
- Não conseguimos alterar os endereços
 - Eles vêm de fábrica
 - E estão “soldados” à memória
- Mas, conseguimos usá-los

Memória

000	001	002
003	004	005
006	007	008

Ponteiros

- Um ponteiro é uma variável que armazena um endereço de memória
- Exemplo. A variável `p` abaixo é um ponteiro e armazena o endereço 003



Ponteiros

- Um ponteiro é uma variável que armazena um endereço de memória
- Exemplo. A variável `p` abaixo é um ponteiro e armazena o endereço 003

p é uma variável do tipo ponteiro.

Portanto, p armazena um endereço.

p armazena o valor 003.

O endereço de p é 007.

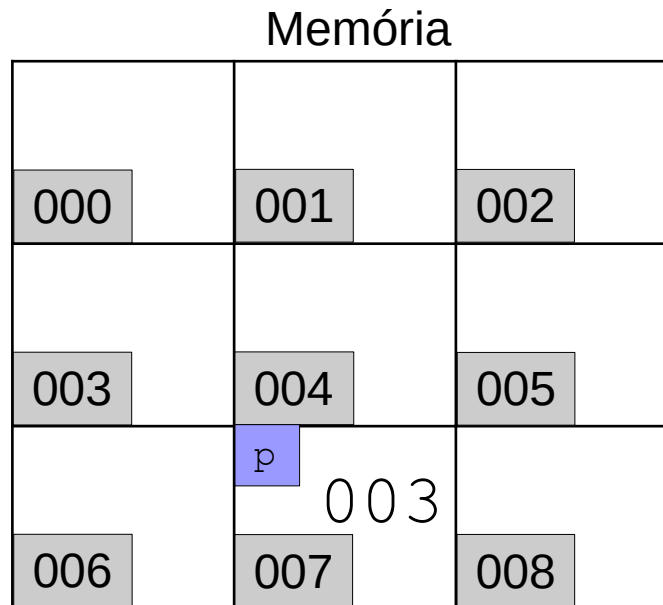
O endereço 003 não está armazenando nada.

Memória

000	001	002
003	004	005
	p 003	
006	007	008

Ponteiros

- Dizemos que p aponta para o endereço 003



Ponteiros

- Motivação para ponteiros
 - Duas ou mais variáveis apontando para mesma posição de memória
 - Uso eficiente da memória para arrays
 - Alocação de memória somente para aquilo em uso
 - Funções
 - Passagem de parâmetros/argumentos por referência
 - Retorno de mais de um resultado
- Requer boa disciplina por parte do programador

Ponteiros

- Capaz de armazenar endereço de outras variáveis

Ponteiros

- Capaz de armazenar endereço de outras variáveis
- Declaração
 - `tipo *identificador;`
 - Exemplo: `int *p;`
 - `p` é uma variável do tipo ponteiro
 - Ou seja, ele armazena um endereço
 - `p` não é uma variável do tipo inteiro
 - O endereço que `p` armazena contém um inteiro

Ponteiros

```
#include <stdio.h>
int main() {
    int *p;
    return 0;
}
```

Execução

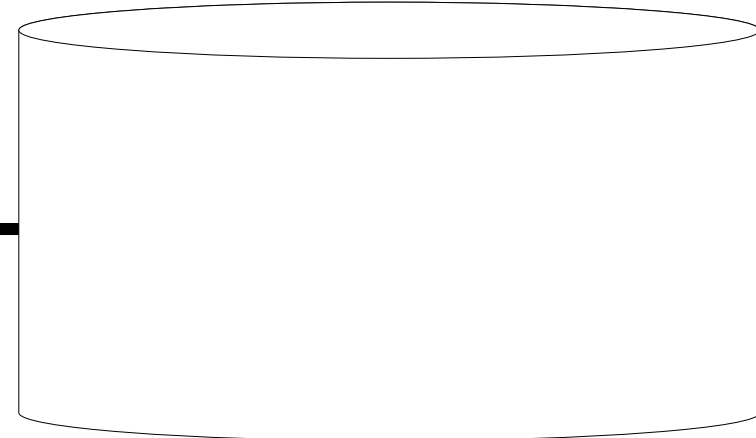
```
#include <stdio.h>
int main() {
    int *p;
    return 0;
}
```

Monitor / Teclado

Memória

000	001	002
003	004	005
006	007	008

HD



Execução

```
#include <stdio.h>
int main() {
    int *p;
    return 0;
}
```

A variável p está pronta para armazenar um endereço de memória. E, neste endereço, será armazenado um inteiro.

Monitor / Teclado

Memória

p		
000	001	002
003	004	005
006	007	008

HD

Execução

```
#include <stdio.h>
int main() {
    int *p;
    return 0;
}
```

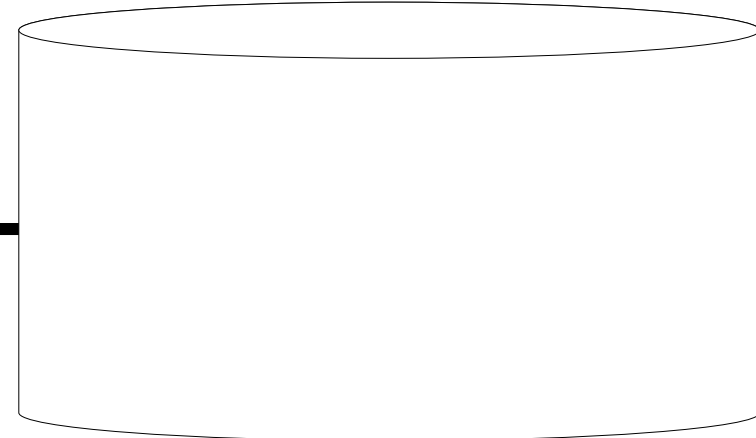


Monitor / Teclado

Memória

000	001	002
003	004	005
006	007	008

HD



Ponteiros

- &
 - Operador de endereço

```
#include<stdio.h>
int main() {
    int x;
    int *p;
    p = &x;
}
```

- `&x` é o endereço da variável `x`

Ponteiros

```
#include <stdio.h>
int main() {
    int *p;
    int x;
    x = 52;
    p = &x;
    printf("Valor de x: %d\n", x);
    printf("Endereco de x: %p\n", &x);
    printf("Valor de p: %p\n", p);
    printf("Endereco de p: %p\n", &p);
    return 0;
}
```

Execução

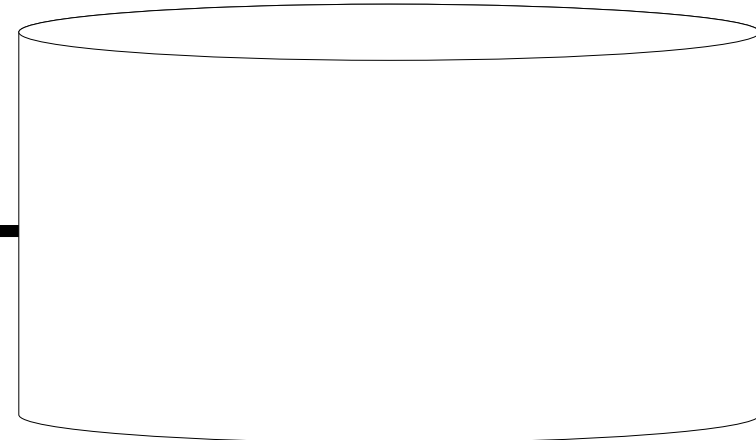
```
#include <stdio.h>
int main() {
    int *p;
    int x;
    x = 52;
    p = &x;
    printf("Valor de x: %d\n", x);
    printf("Endereco de x: %p\n", &x);
    printf("Valor de p: %p\n", p);
    printf("Endereco de p: %p\n", &p);
    return 0;
}
```

Monitor / Teclado

Memória

000	001	002
003	004	005
006	007	008

HD



Execução

```
#include <stdio.h>
int main() {
    int *p;
    int x;
    x = 52;
    p = &x;
    printf("Valor de x: %d\n", x);
    printf("Endereco de x: %p\n", &x);
    printf("Valor de p: %p\n", p);
    printf("Endereco de p: %p\n", &p);
    return 0;
}
```

A variável p está pronta para armazenar um endereço de memória.

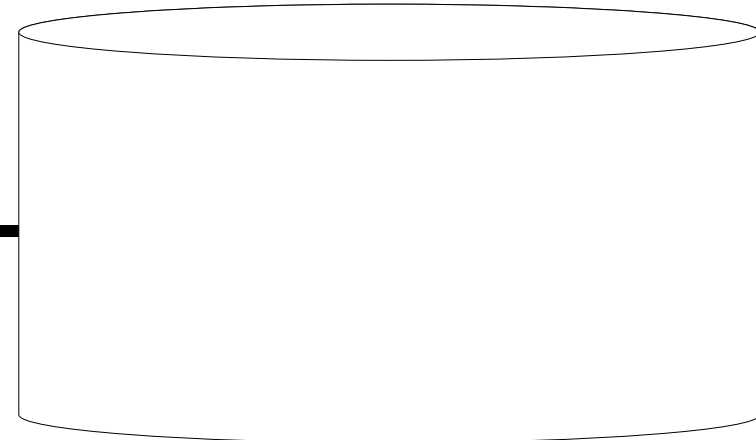
E, neste endereço, será armazenado um inteiro.

Monitor / Teclado

Memória

p		
000	001	002
003	004	005
006	007	008

HD

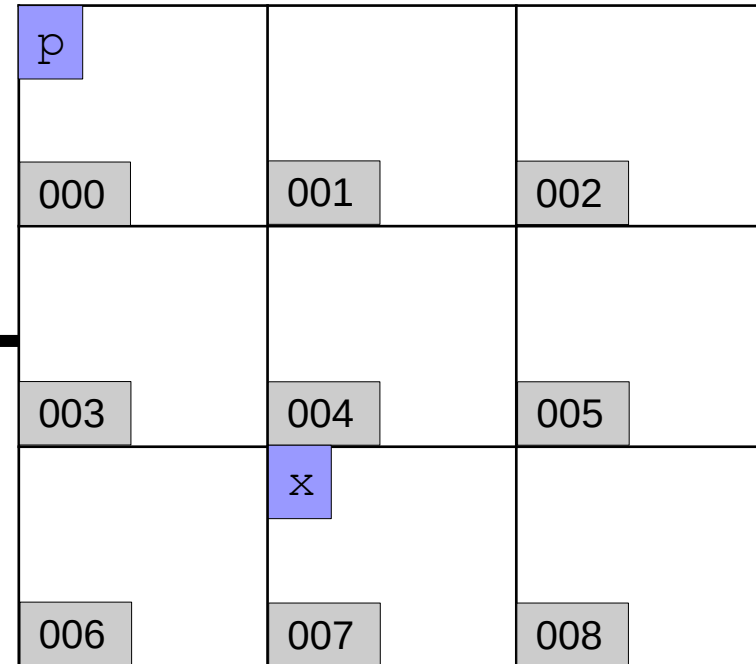


Execução

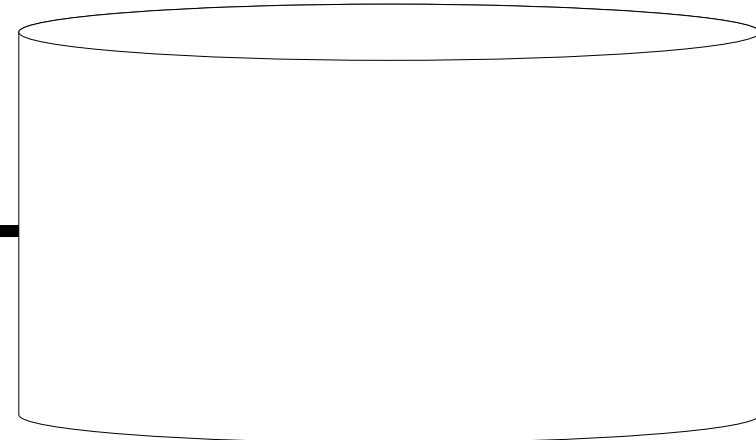
```
#include <stdio.h>
int main() {
    int *p;
    int x;
    x = 52;
    p = &x;
    printf("Valor de x: %d\n", x);
    printf("Endereco de x: %p\n", &x);
    printf("Valor de p: %p\n", p);
    printf("Endereco de p: %p\n", &p);
    return 0;
}
```

Monitor / Teclado

Memória



HD

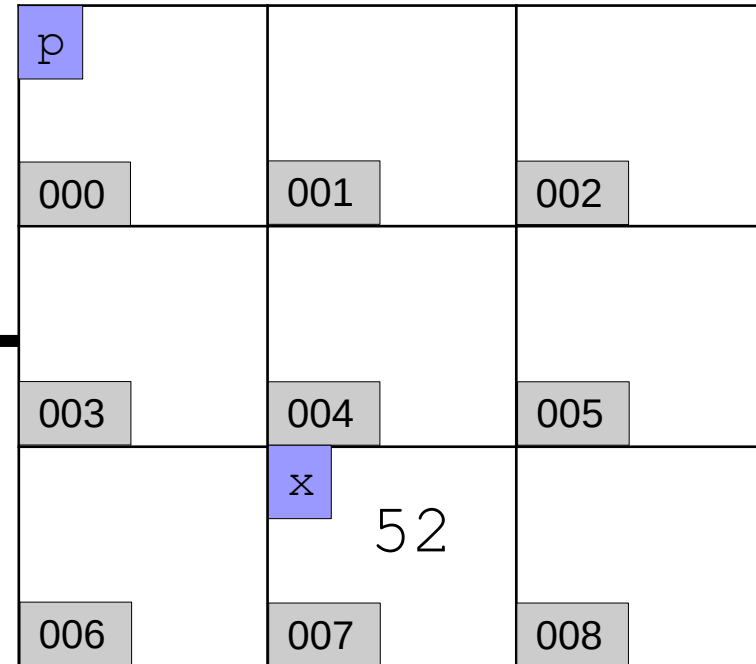


Execução

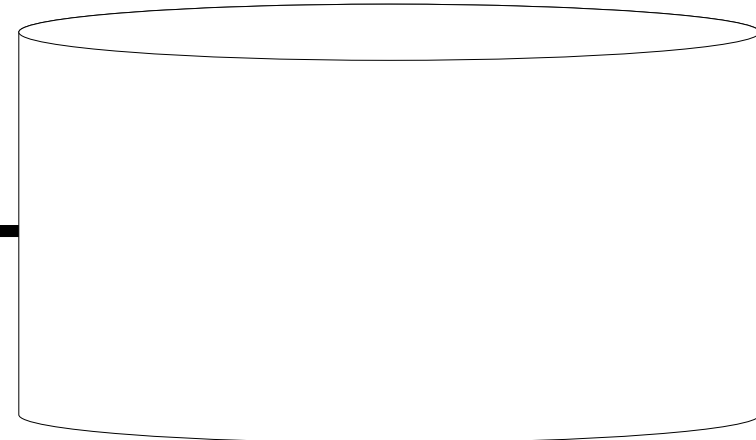
```
#include <stdio.h>
int main() {
    int *p;
    int x;
    x = 52;
    p = &x;
    printf("Valor de x: %d\n", x);
    printf("Endereco de x: %p\n", &x);
    printf("Valor de p: %p\n", p);
    printf("Endereco de p: %p\n", &p);
    return 0;
}
```

Monitor / Teclado

Memória



HD



Execução

```
#include <stdio.h>
int main() {
    int *p;
    int x;
    x = 52;
    p = &x;
    printf("Valor de x: %d\n", x);
    printf("Endereco de x: %p\n", &x);
    printf("Valor de p: %p\n", p);
    printf("Endereco de p: %p\n", &p);
    return 0;
}
```

*&x faz o computador
procurar e retornar o
endereço de x, que é 007.*

Monitor / Teclado

Memória

p		
000	001	002
003	004	005
	x	
006	007	008

52

HD

Execução

```
#include <stdio.h>
int main() {
    int *p;
    int x;
    x = 52;
    p = &x;
    printf("Valor de x: %d\n", x);
    printf("Endereco de x: %p\n", &x);
    printf("Valor de p: %p\n", p);
    printf("Endereco de p: %p\n", &p);
    return 0;
}
```

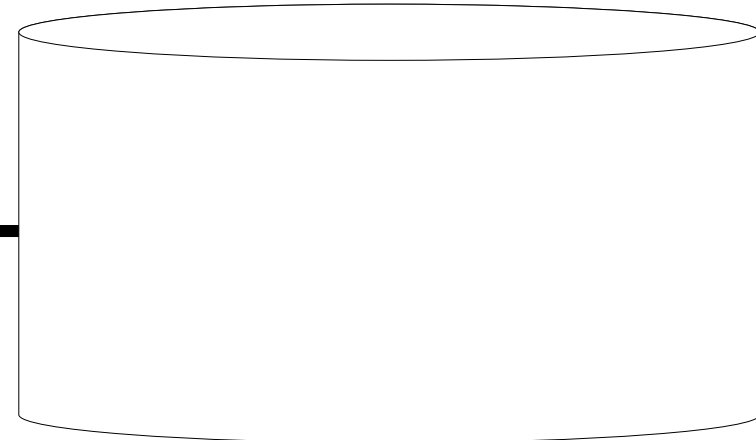
&x faz o computador procurar e retornar o endereço de x, que é 007.

Monitor / Teclado

Memória

p		
007		
000	001	002
003	004	005
	x	
	52	
006	007	008

HD



Execução

```
#include <stdio.h>
int main() {
    int *p;
    int x;
    x = 52;
    p = &x;
    printf("Valor de x: %d\n", x);
    printf("Endereco de x: %p\n", &x);
    printf("Valor de p: %p\n", p);
    printf("Endereco de p: %p\n", &p);
    return 0;
}
```

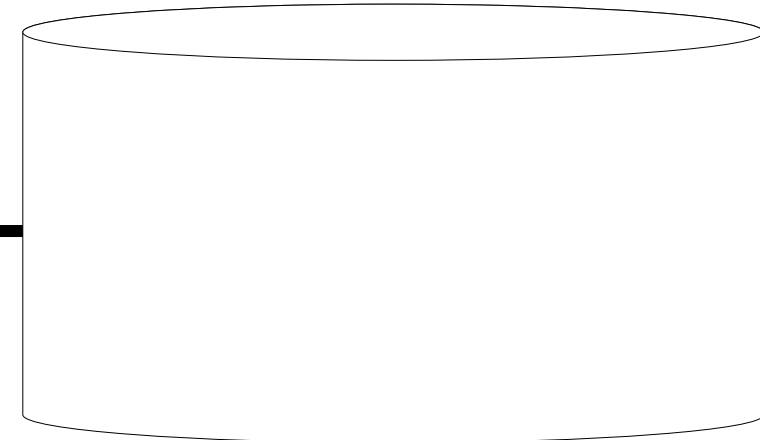
Monitor / Teclado

Valor de x: 52

Memória

p		
007		
000	001	002
003	004	005
	x	
	52	
006	007	008

HD



Execução

```
#include <stdio.h>
int main() {
    int *p;
    int x;
    x = 52;
    p = &x;
    printf("Valor de x: %d\n", x);
    printf("Endereco de x: %p\n", &x);
    printf("Valor de p: %p\n", p);
    printf("Endereco de p: %p\n", &p);
    return 0;
}
```

%p diz ao printf() que é para imprimir um endereço de memória.

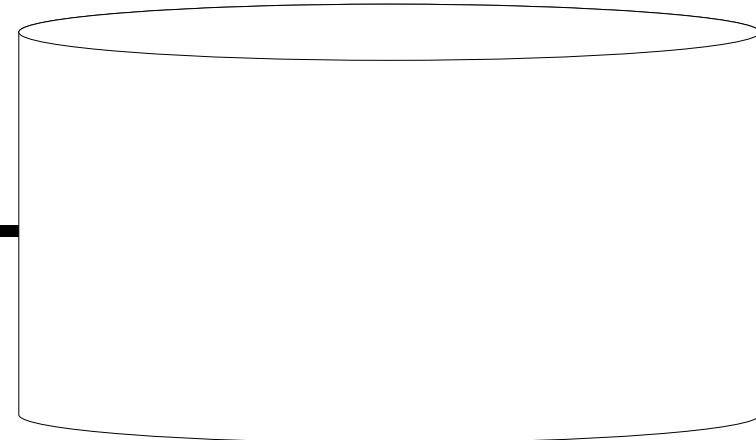
Monitor / Teclado

Valor de x: 52
Endereco de x: 007

Memória

p		
007		
000	001	002
003	004	005
	x	
	52	
006	007	008

HD



Execução

```
#include <stdio.h>
int main() {
    int *p;
    int x;
    x = 52;
    p = &x;
    printf("Valor de x: %d\n", x);
    printf("Endereco de x: %p\n", &x);
    printf("Valor de p: %p\n", p);
    printf("Endereco de p: %p\n", &p);
    return 0;
}
```



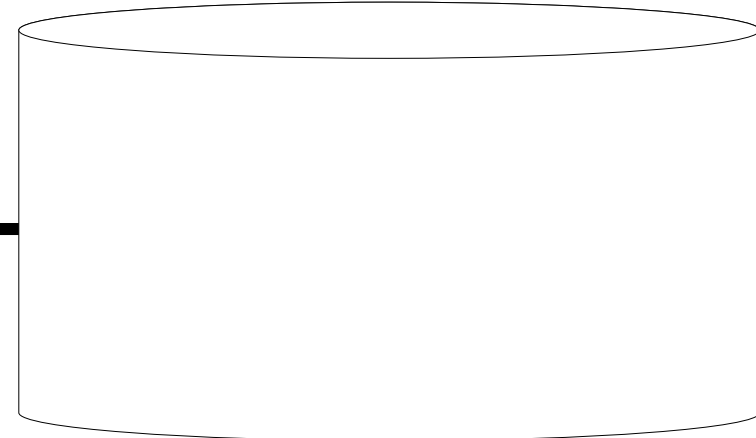
Monitor / Teclado

Valor de x: 52
Endereco de x: 007
Valor de p: 007

Memória

p		
007		
000	001	002
003	004	005
	x	
	52	
006	007	008

HD



Execução

```
#include <stdio.h>
int main() {
    int *p;
    int x;
    x = 52;
    p = &x;
    printf("Valor de x: %d\n", x);
    printf("Endereco de x: %p\n", &x);
    printf("Valor de p: %p\n", p);
    printf("Endereco de p: %p\n", &p);
    return 0;
}
```

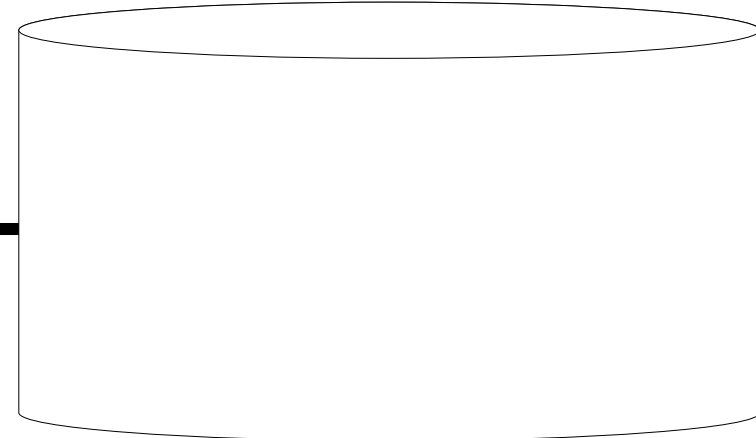
Monitor / Teclado

Valor de x: 52
Endereco de x: 007
Valor de p: 007
Endereco de p: 000

Memória

p		
007		
000	001	002
003	004	005
	x	
	52	
006	007	008

HD



Execução

```
#include <stdio.h>
int main() {
    int *p;
    int x;
    x = 52;
    p = &x;
    printf("Valor de x: %d\n", x);
    printf("Endereco de x: %p\n", &x);
    printf("Valor de p: %p\n", p);
    printf("Endereco de p: %p\n", &p);
    return 0;
}
```

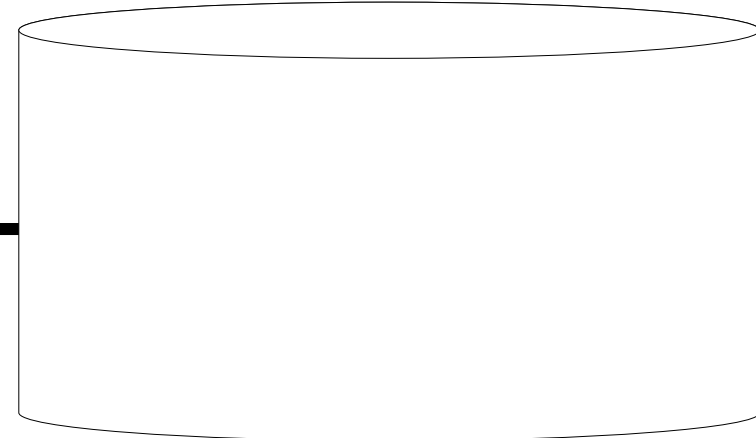


Monitor / Teclado

Memória

000	001	002
003	004	005
006	007	008

HD



Ponteiros

- *

- Operador de conteúdo

```
#include<stdio.h>
```

```
int main() {
```

```
    int x;
```

```
    int *p;
```

```
    p = &x;
```

```
    *p = 30;
```

```
}
```

- *Atribui um valor no endereço armazenado por p*
- *Altera o valor no endereço armazenado por p*
- *Não altera p*
- **p = 30; é equivalente a x = 30;*

- &x é o endereço da variável x

- *p é a variável x

Execução

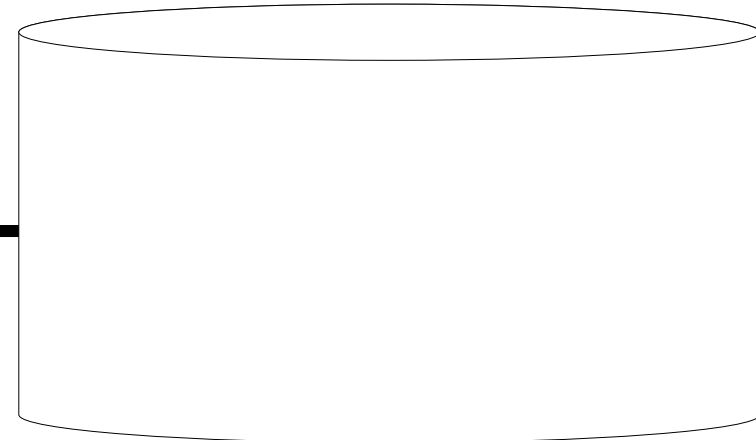
```
#include<stdio.h>
int main() {
    int x;
    int *p;
    p = &x;
    *p = 30;
}
```

Monitor / Teclado

Memória

000	001	002
003	004	005
006	007	008

HD



Execução

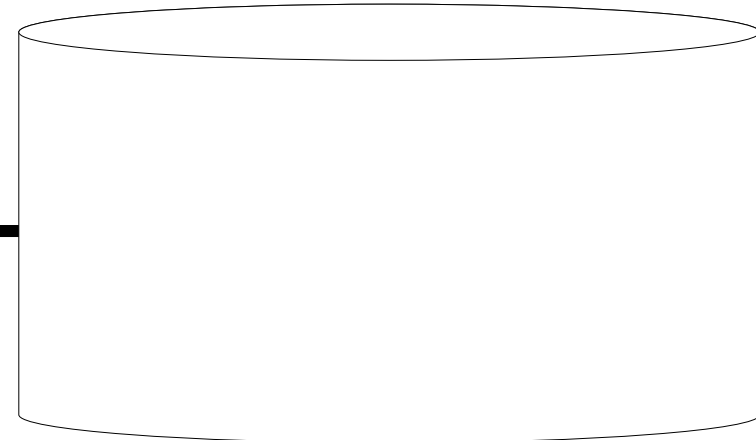
```
#include<stdio.h>
int main() {
    → int x;
      int *p;
      p = &x;
      *p = 30;
}
```

Monitor / Teclado

Memória

x		
000	001	002
003	004	005
006	007	008

HD



Execução

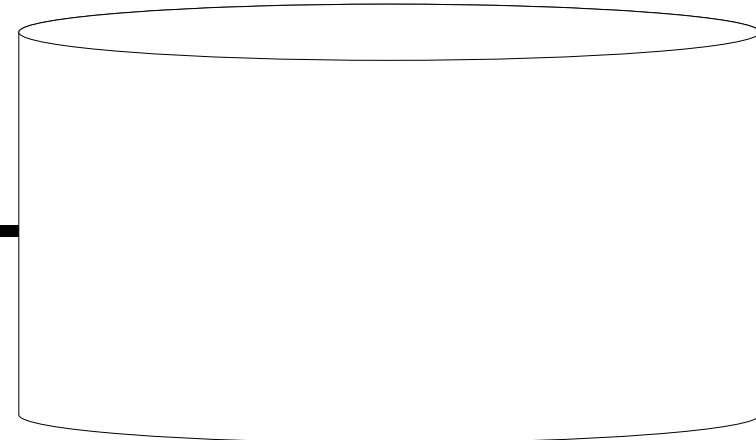
```
#include<stdio.h>
int main() {
    int x;
    → int *p;
    p = &x;
    *p = 30;
}
```

Monitor / Teclado

Memória

x	p	
000	001	002
003	004	005
006	007	008

HD



Execução

```
#include<stdio.h>
int main() {
    int x;
    int *p;
    → p = &x;
    *p = 30;
}
```

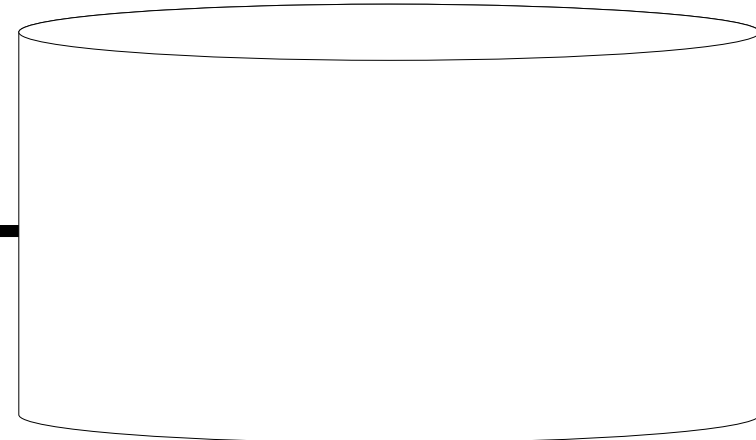
*&x é o endereço da
variável x, ou seja,
&x é 000.*

Monitor / Teclado

Memória

x	p	
000	001	002
003	004	005
006	007	008

HD



Execução

```
#include<stdio.h>
int main() {
    int x;
    int *p;
    → p = &x;
      *p = 30;
}
```

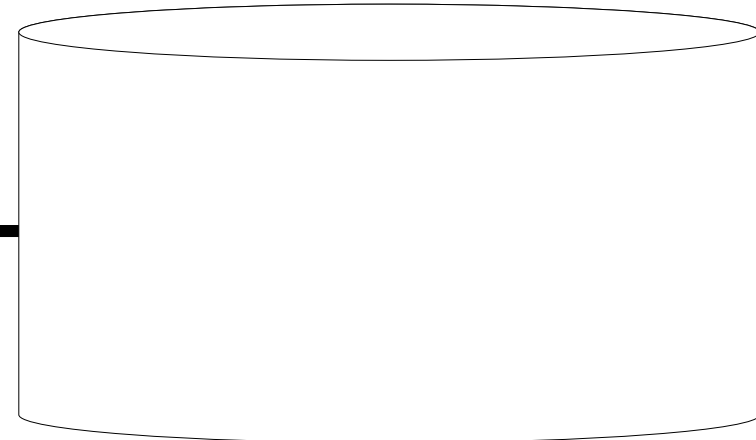
*&x é o endereço da
variável x, ou seja,
&x é 000.*

Monitor / Teclado

Memória

x	p	
000	000	002
001		
003	004	005
006	007	008

HD



Execução

```
#include<stdio.h>
int main() {
    int x;
    int *p;
    p = &x;
    → *p = 30;
}
```

*Ao encontrar *p, o computador:*

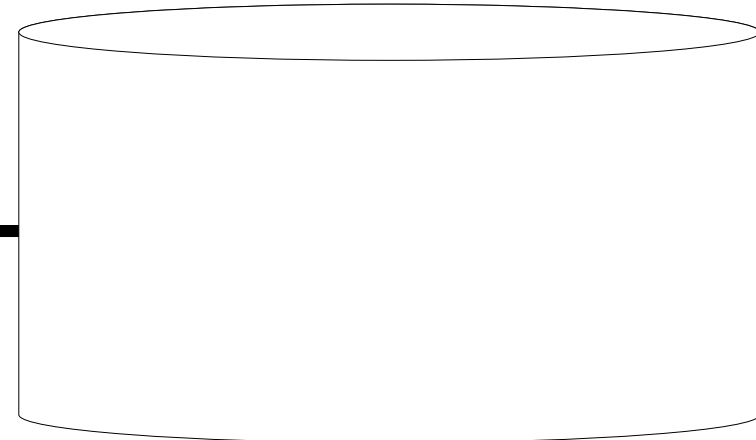
- 1. Olha o endereço armazenado por p (no caso, 000)*
- 2. Vai neste endereço*
- 3. Grava 30 neste endereço*

Monitor / Teclado

Memória

x	p	
000	000	002
003	004	005
006	007	008

HD



Execução

```
#include<stdio.h>
int main() {
    int x;
    int *p;
    p = &x;
    → *p = 30;
}
```

*Ao encontrar *p, o computador:*

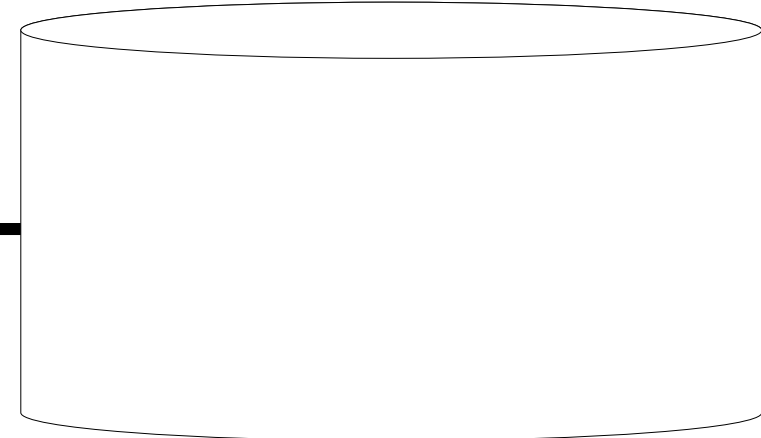
- 1. Olha o endereço armazenado por p (no caso, 000)*
- 2. Vai neste endereço*
- 3. Grava 30 neste endereço*

Monitor / Teclado

Memória

x	p	
30	000	
000	001	002
003	004	005
006	007	008

HD



Execução

```
#include<stdio.h>
int main() {
    int x;
    int *p;
    p = &x;
    *p = 30;
}
```

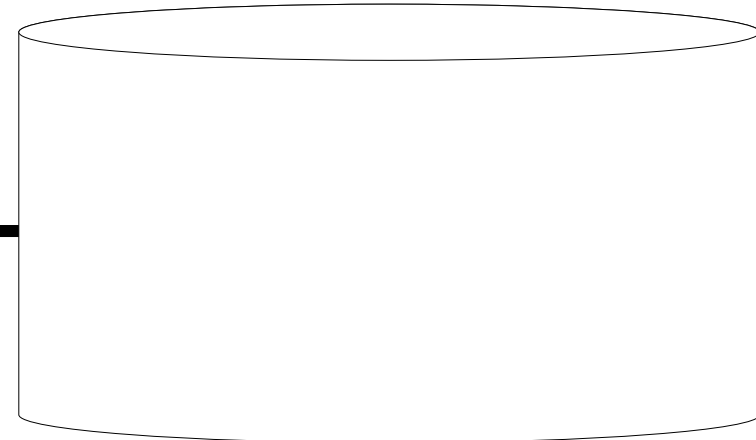


Monitor / Teclado

Memória

000	001	002
003	004	005
006	007	008

HD



Ponteiros

```
#include <stdio.h>
int main() {
    int *p;
    int x;
    x = 52;
    printf("Valor de x: %d\n", x);
    p = &x;
    *p = 35;
    printf("Valor de x: %d\n", x);
    printf("Endereco de x: %p\n", &x);
    printf("Valor de p: %p\n", p);
    printf("Endereco de p: %p\n", &p);
    return 0;
}
```

Execução

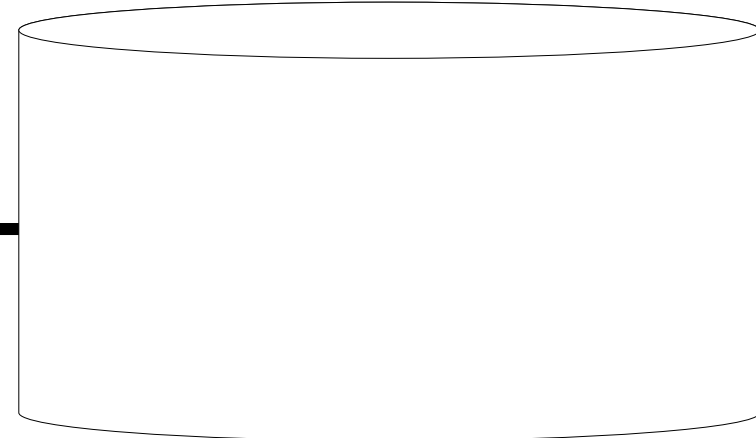
```
#include <stdio.h>
int main() {
    int *p;
    int x;
    x = 52;
    printf("Valor de x: %d\n", x);
    p = &x;
    *p = 35;
    printf("Valor de x: %d\n", x);
    printf("Endereco de x: %p\n", &x);
    printf("Valor de p: %p\n", p);
    printf("Endereco de p: %p\n", &p);
    return 0;
}
```

Monitor / Teclado

Memória

000	001	002
003	004	005
006	007	008

HD



Execução

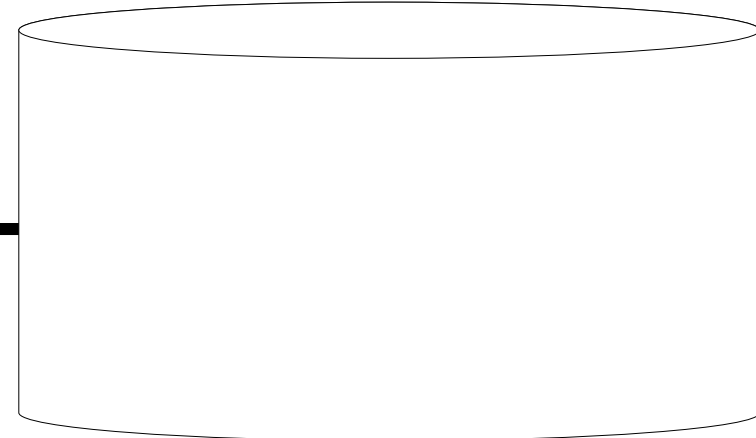
```
#include <stdio.h>
int main() {
    int *p;
    int x;
    x = 52;
    printf("Valor de x: %d\n", x);
    p = &x;
    *p = 35;
    printf("Valor de x: %d\n", x);
    printf("Endereco de x: %p\n", &x);
    printf("Valor de p: %p\n", p);
    printf("Endereco de p: %p\n", &p);
    return 0;
}
```

Monitor / Teclado

Memória

p		
000	001	002
003	004	005
006	007	008

HD



Execução

```
#include <stdio.h>
int main() {
    int *p;
    int x;
    x = 52;
    printf("Valor de x: %d\n", x);
    p = &x;
    *p = 35;
    printf("Valor de x: %d\n", x);
    printf("Endereco de x: %p\n", &x);
    printf("Valor de p: %p\n", p);
    printf("Endereco de p: %p\n", &p);
    return 0;
}
```

Monitor / Teclado

Memória

p	x	
000	001	002
003	004	005
006	007	008

HD

Execução

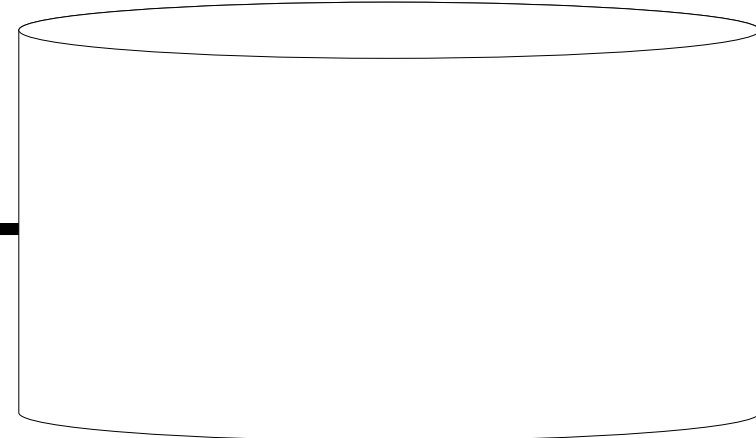
```
#include <stdio.h>
int main() {
    int *p;
    int x;
    x = 52;
    printf("Valor de x: %d\n", x);
    p = &x;
    *p = 35;
    printf("Valor de x: %d\n", x);
    printf("Endereco de x: %p\n", &x);
    printf("Valor de p: %p\n", p);
    printf("Endereco de p: %p\n", &p);
    return 0;
}
```

Monitor / Teclado

Memória

p	x	
000	001	002
	52	
003	004	005
006	007	008

HD



Execução

```
#include <stdio.h>
int main() {
    int *p;
    int x;
    x = 52;
    printf("Valor de x: %d\n", x);
    p = &x;
    *p = 35;
    printf("Valor de x: %d\n", x);
    printf("Endereco de x: %p\n", &x);
    printf("Valor de p: %p\n", p);
    printf("Endereco de p: %p\n", &p);
    return 0;
}
```

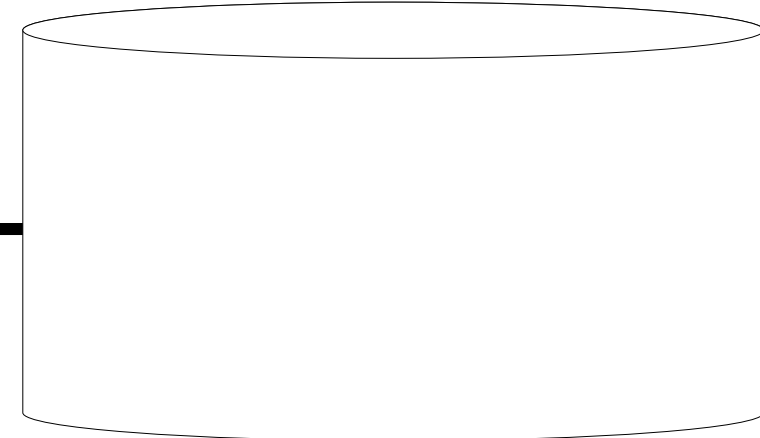
Monitor / Teclado

Valor de x: 52

Memória

p	x	
000	001	002
003	004	005
006	007	008

HD



Execução

```
#include <stdio.h>
int main() {
    int *p;
    int x;
    x = 52;
    printf("Valor de x: %d\n", x);
    p = &x;
    *p = 35;
    printf("Valor de x: %d\n", x);
    printf("Endereco de x: %p\n", &x);
    printf("Valor de p: %p\n", p);
    printf("Endereco de p: %p\n", &p);
    return 0;
}
```

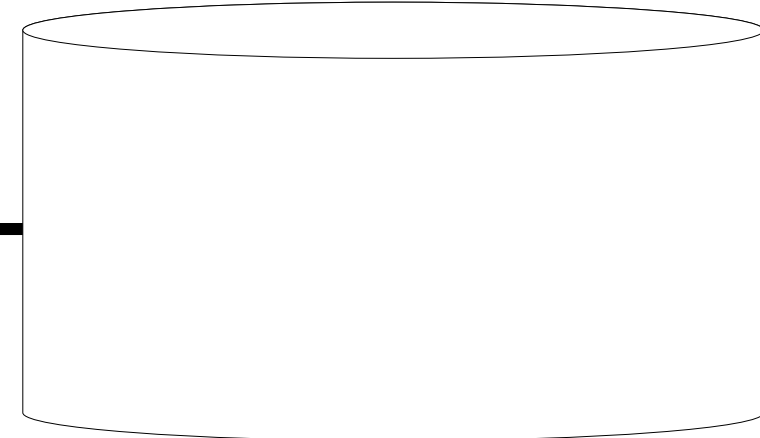
Monitor / Teclado

Valor de x: 52

Memória

p	x	
001	52	
000	001	002
003	004	005
006	007	008

HD



Execução

```
#include <stdio.h>
int main() {
    int *p;
    int x;
    x = 52;
    printf("Valor de x: %d\n", x);
    p = &x;
    *p = 35;
    printf("Valor de x: %d\n", x);
    printf("Endereco de x: %p\n", &x);
    printf("Valor de p: %p\n", p);
    printf("Endereco de p: %p\n", &p);
    return 0;
}
```

1. O computador vai até *p*;
2. Pega o endereço armazenado em *p* (001)
3. Vai até 001
4. Grava 35 lá em 001

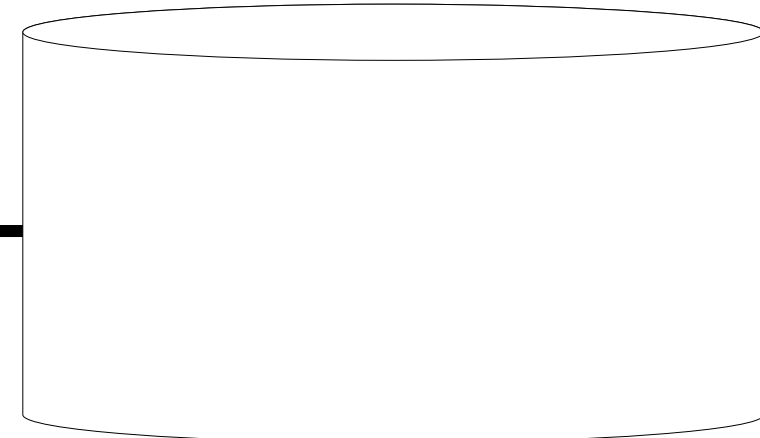
Monitor / Teclado

Valor de x: 52

Memória

p	x	
001	35	
000	001	002
003	004	005
006	007	008

HD



Execução

```
#include <stdio.h>
int main() {
    int *p;
    int x;
    x = 52;
    printf("Valor de x: %d\n", x);
    p = &x;
    *p = 35;
    printf("Valor de x: %d\n", x);
    printf("Endereco de x: %p\n", &x);
    printf("Valor de p: %p\n", p);
    printf("Endereco de p: %p\n", &p);
    return 0;
}
```

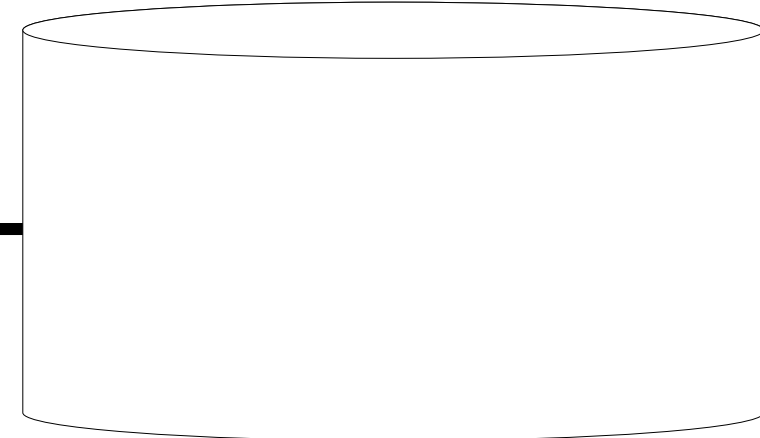
Monitor / Teclado

Valor de x: 52
Valor de x: 35

Memória

p	x	
001	35	
000	001	002
003	004	005
006	007	008

HD



Execução

```
#include <stdio.h>
int main() {
    int *p;
    int x;
    x = 52;
    printf("Valor de x: %d\n", x);
    p = &x;
    *p = 35;
    printf("Valor de x: %d\n", x);
    printf("Endereco de x: %p\n", &x);
    printf("Valor de p: %p\n", p);
    printf("Endereco de p: %p\n", &p);
    return 0;
}
```

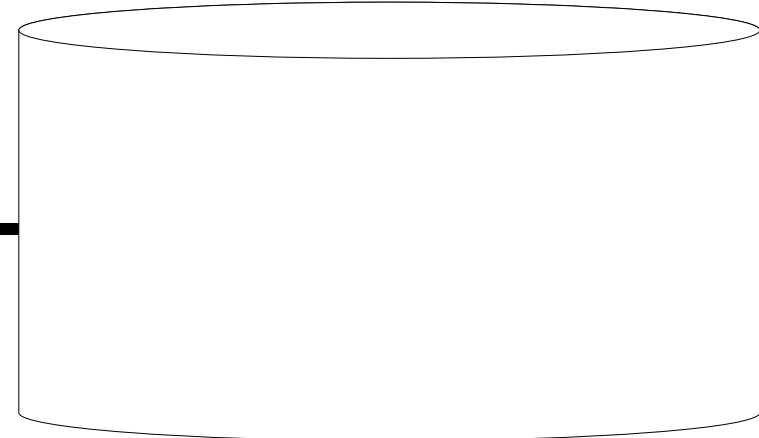
Monitor / Teclado

Valor de x: 52
Valor de x: 35
Endereco de x: 001

Memória

p	x	
001	35	
000	001	002
003	004	005
006	007	008

HD



Execução

```
#include <stdio.h>
int main() {
    int *p;
    int x;
    x = 52;
    printf("Valor de x: %d\n", x);
    p = &x;
    *p = 35;
    printf("Valor de x: %d\n", x);
    printf("Endereco de x: %p\n", &x);
    printf("Valor de p: %p\n", p);
    printf("Endereco de p: %p\n", &p);
    return 0;
}
```

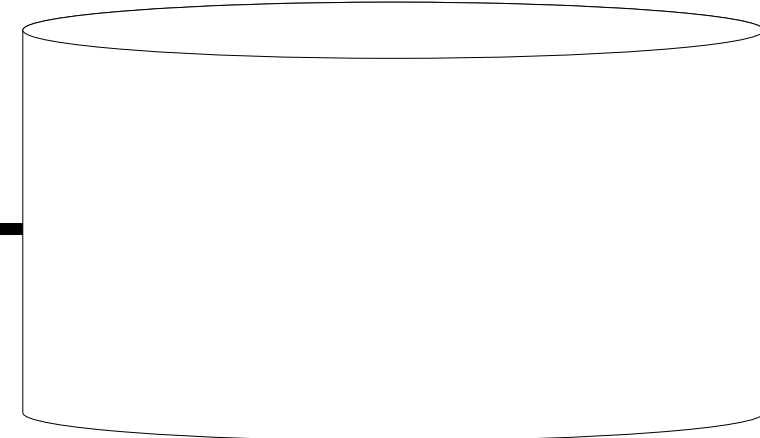
Monitor / Teclado

Valor de x: 52
Valor de x: 35
Endereco de x: 001
Valor de p: 001

Memória

p	x	
001	35	
000	001	002
003	004	005
006	007	008

HD



Execução

```
#include <stdio.h>
int main() {
    int *p;
    int x;
    x = 52;
    printf("Valor de x: %d\n", x);
    p = &x;
    *p = 35;
    printf("Valor de x: %d\n", x);
    printf("Endereco de x: %p\n", &x);
    printf("Valor de p: %p\n", p);
    printf("Endereco de p: %p\n", &p);
    return 0;
}
```

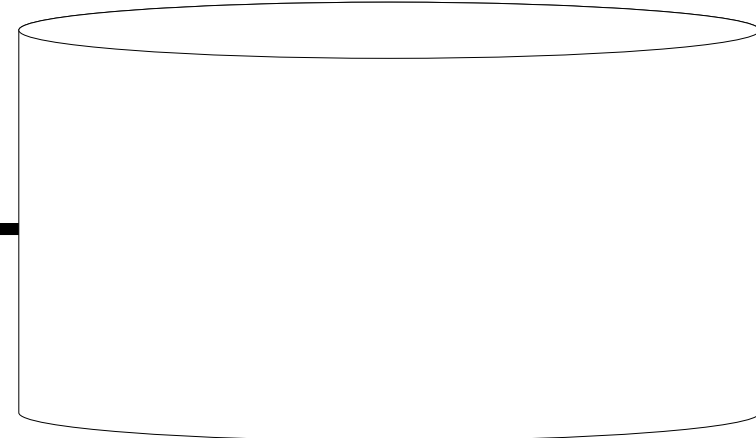
Monitor / Teclado

Valor de x: 35
Endereco de x: 001
Valor de p: 001
Endereco de p: 000

Memória

p	x	
001	35	
000	001	002
003	004	005
006	007	008

HD



Execução

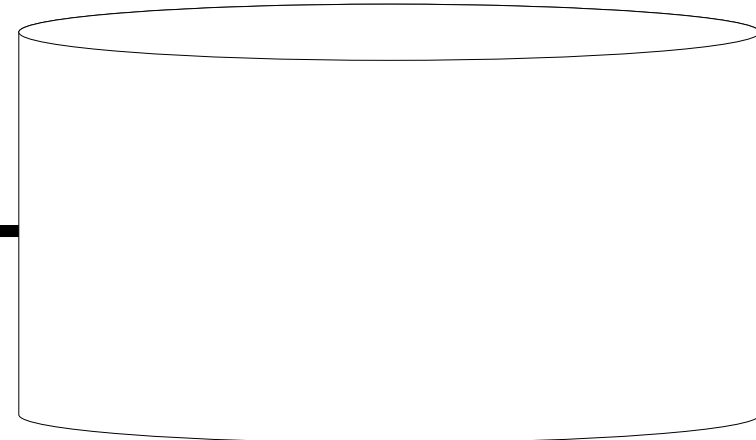
```
#include <stdio.h>
int main() {
    int *p;
    int x;
    x = 52;
    printf("Valor de x: %d\n", x);
    p = &x;
    *p = 35;
    printf("Valor de x: %d\n", x);
    printf("Endereco de x: %p\n", &x);
    printf("Valor de p: %p\n", p);
    printf("Endereco de p: %p\n", &p);
    return 0;
}
```

Monitor / Teclado

Memória

000	001	002
003	004	005
006	007	008

HD



Ponteiros

```
#include <stdio.h>
int main() {
    int x;
    int* p;
    x = 10;
    p = &x;
    *p = *p + 7;
    printf("x = %d \n", x);
    printf("p = %p \n", p);
    printf("*p = %d \n", *p);
    printf("&x = %p \n", &x);
    printf("&p = %p \n", &p);
    return 0;
}
```

Execução

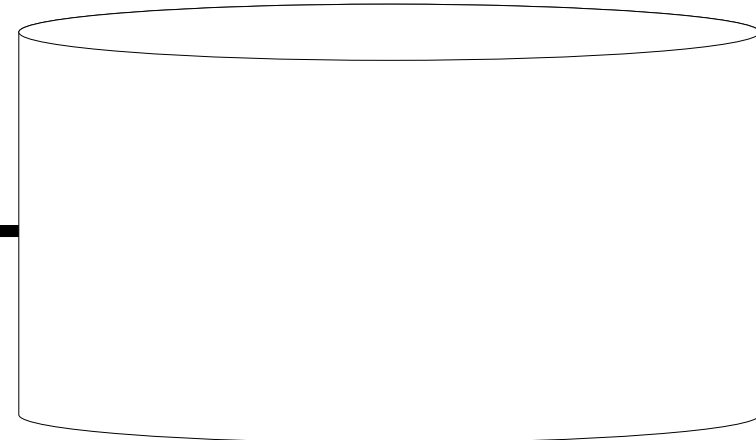
```
#include <stdio.h>
int main() {
    int x;
    int* p;
    x = 10;
    p = &x;
    *p = *p + 7;
    printf("x = %d \n", x);
    printf("p = %p \n", p);
    printf("*p = %d \n", *p);
    printf("&x = %p \n", &x);
    printf("&p = %p \n", &p);
    return 0;
}
```

Monitor / Teclado

Memória

000	001	002
003	004	005
006	007	008

HD



Execução

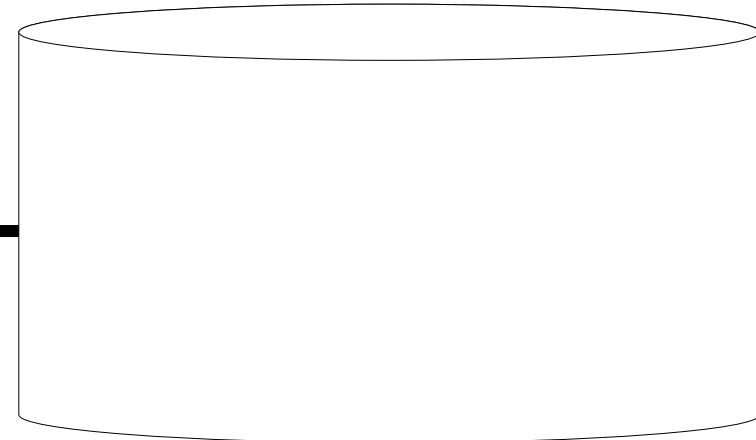
```
#include <stdio.h>
int main() {
    int x;
    int* p;
    x = 10;
    p = &x;
    *p = *p + 7;
    printf("x = %d \n", x);
    printf("p = %p \n", p);
    printf("*p = %d \n", *p);
    printf("&x = %p \n", &x);
    printf("&p = %p \n", &p);
    return 0;
}
```

Monitor / Teclado

Memória

x		
000	001	002
003	004	005
006	007	008

HD



Execução

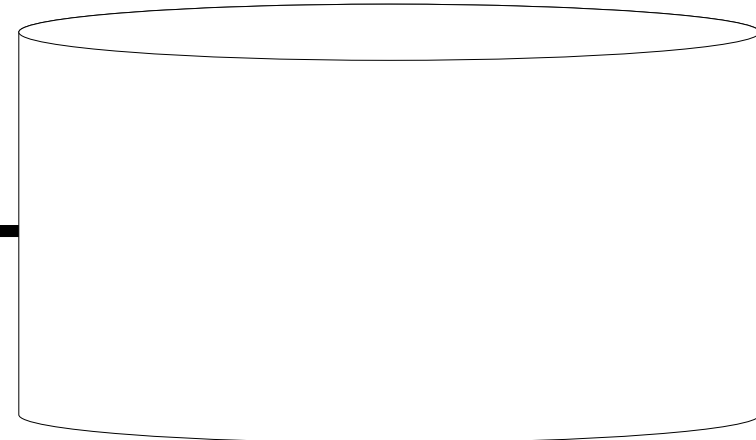
```
#include <stdio.h>
int main() {
    int x;
    int* p;
    x = 10;
    p = &x;
    *p = *p + 7;
    printf("x = %d \n", x);
    printf("p = %p \n", p);
    printf("*p = %d \n", *p);
    printf("&x = %p \n", &x);
    printf("&p = %p \n", &p);
    return 0;
}
```

Monitor / Teclado

Memória

x	p	
000	001	002
003	004	005
006	007	008

HD



Execução

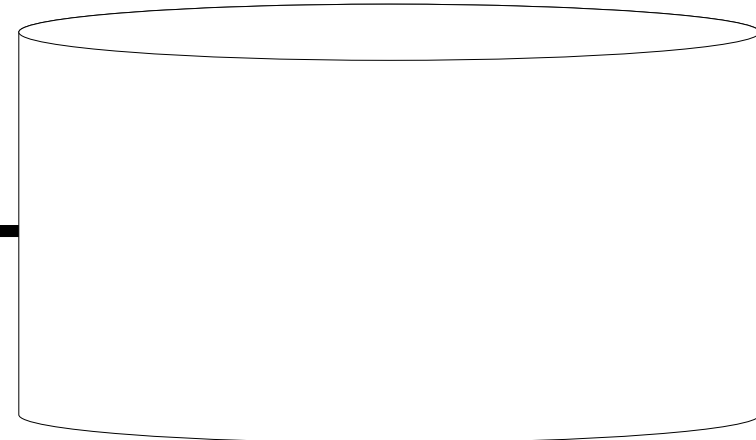
```
#include <stdio.h>
int main() {
    int x;
    int* p;
    x = 10;
    p = &x;
    *p = *p + 7;
    printf("x = %d \n", x);
    printf("p = %p \n", p);
    printf("*p = %d \n", *p);
    printf("&x = %p \n", &x);
    printf("&p = %p \n", &p);
    return 0;
}
```

Monitor / Teclado

Memória

x	p	
10		
000	001	002
003	004	005
006	007	008

HD



Execução

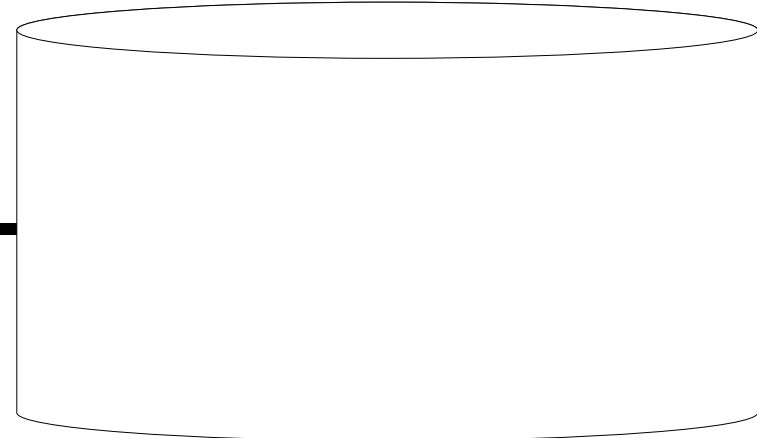
```
#include <stdio.h>
int main() {
    int x;
    int* p;
    x = 10;
    → p = &x;
    *p = *p + 7;
    printf("x = %d \n", x);
    printf("p = %p \n", p);
    printf("*p = %d \n", *p);
    printf("&x = %p \n", &x);
    printf("&p = %p \n", &p);
    return 0;
}
```

Monitor / Teclado

Memória

x	10	p	000	
000		001		002
003		004		005
006		007		008

HD



Execução

```
#include <stdio.h>
int main() {
    int x;
    int* p;
    x = 10;
    p = &x;
    → *p = *p + 7;
    printf("x = %d \n", x);
    printf("p = %p \n", p);
    printf("*p = %d \n", *p);
    printf("&x = %p \n", &x);
    printf("&p = %p \n", &p);
    return 0;
}
```

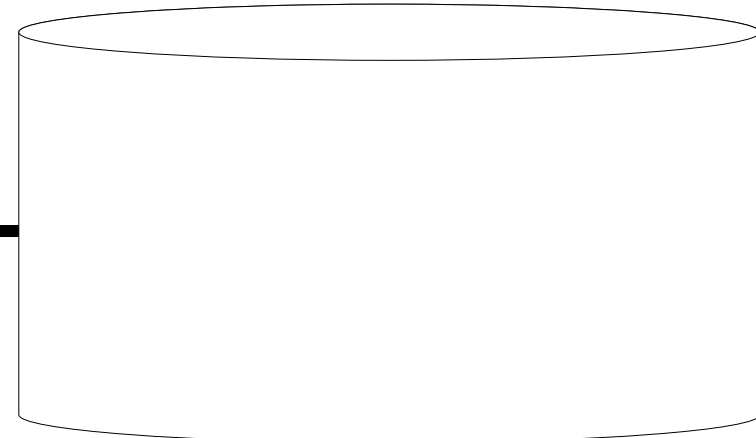
1. O computador vai até *p*;
2. Pega o endereço armazenado em *p* (000)
3. Vai até 000
4. Retorna para a expressão o 10

Monitor / Teclado

Memória

x	p	
10	000	
000	001	002
003	004	005
006	007	008

HD



Execução

```
#include <stdio.h>
int main() {
    int x;
    int* p;
    x = 10;
    p = &x;
    *p = *p + 7;
    printf("x = %d \n", x);
    printf("p = %p \n", p);
    printf("*p = %d \n", *p);
    printf("&x = %p \n", &x);
    printf("&p = %p \n", &p);
    return 0;
}
```

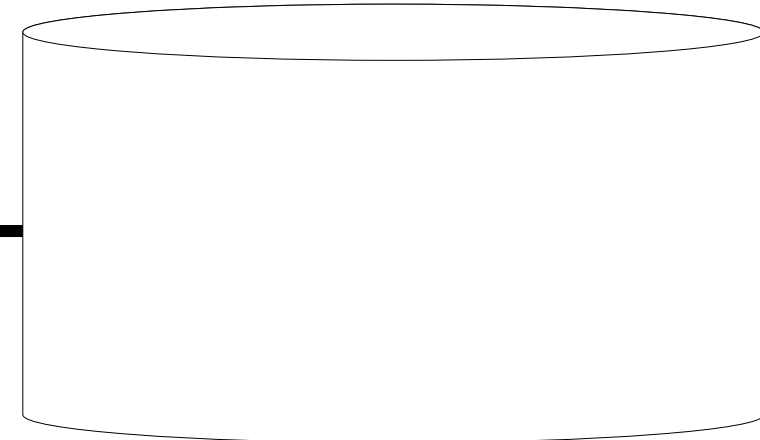
10

Monitor / Teclado

Memória

x	10	p	000	
000		001		002
003		004		005
006		007		008

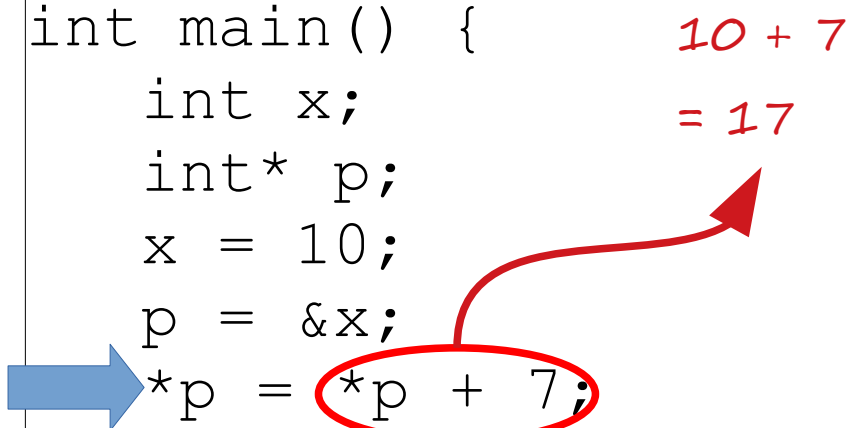
HD



Execução

```
#include <stdio.h>
int main() {
    int x;
    int* p;
    x = 10;
    p = &x;
    *p = *p + 7;
    printf("x = %d \n", x);
    printf("p = %p \n", p);
    printf("*p = %d \n", *p);
    printf("&x = %p \n", &x);
    printf("&p = %p \n", &p);
    return 0;
}
```

*10 + 7
= 17*

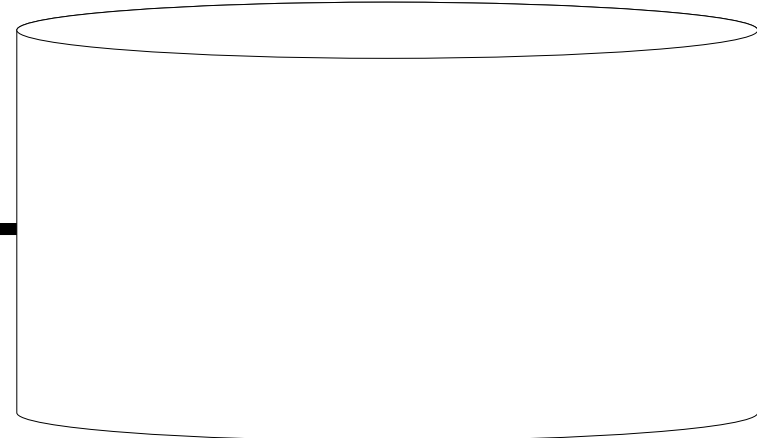


Monitor / Teclado

Memória

x	p	
10	000	
000	001	002
003	004	005
006	007	008

HD



Execução

```
#include <stdio.h>
int main() {
    int x;
    int* p;
    x = 10;
    p = &x;
    → *p = *p + 7;
    printf("x = %d \n", x);
    printf("p = %p \n", p);
    printf("*p = %d \n", *p);
    printf("&x = %p \n", &x);
    printf("&p = %p \n", &p);
    return 0;
}
```

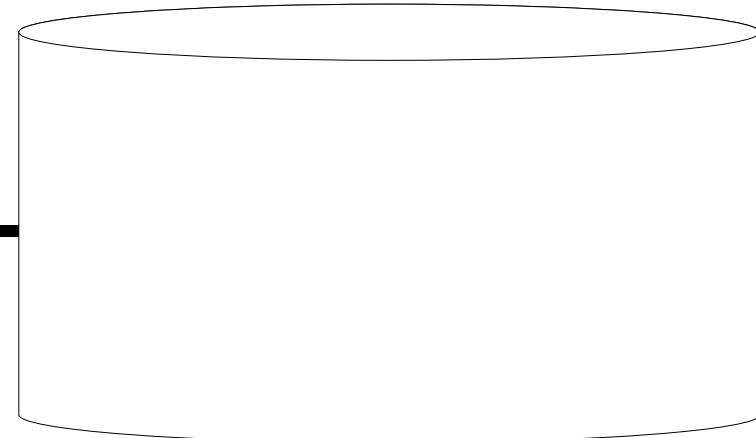
1. O computador vai até *p*;
2. Pega o endereço armazenado em *p* (000)
3. Vai até 000
4. Grava 17 no endereço 000

Monitor / Teclado

Memória

x	17	p	000	
000		001		002
003		004		005
006		007		008

HD



Execução

```
#include <stdio.h>
int main() {
    int x;
    int* p;
    x = 10;
    p = &x;
    *p = *p + 7;
    printf("x = %d \n", x);
    printf("p = %p \n", p);
    printf("*p = %d \n", *p);
    printf("&x = %p \n", &x);
    printf("&p = %p \n", &p);
    return 0;
}
```

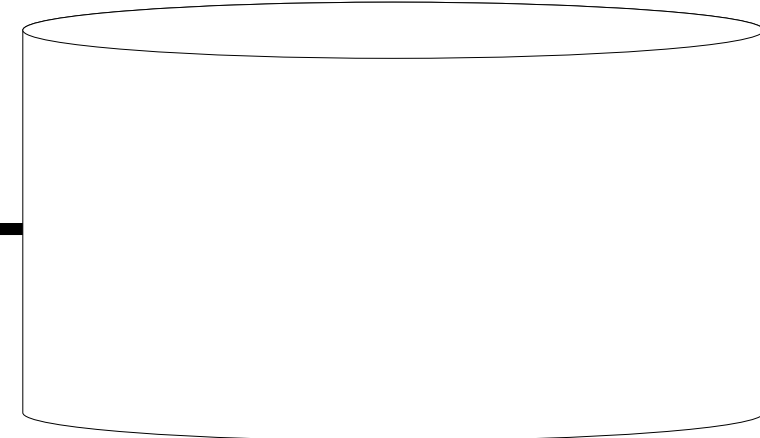
Monitor / Teclado

x = 17

Memória

x	17	p	000	
000		001		002
003		004		005
006		007		008

HD



Execução

```
#include <stdio.h>
int main() {
    int x;
    int* p;
    x = 10;
    p = &x;
    *p = *p + 7;
    printf("x = %d \n", x);
    printf("p = %p \n", p);
    printf("*p = %d \n", *p);
    printf("&x = %p \n", &x);
    printf("&p = %p \n", &p);
    return 0;
}
```

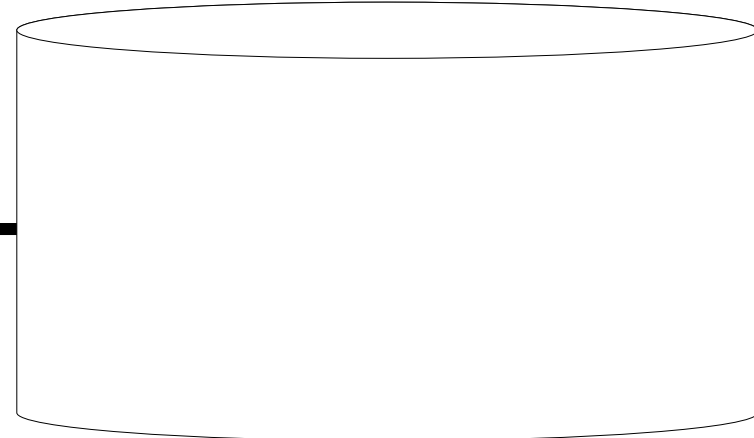
Monitor / Teclado

x = 17
p = 000

Memória

x	17	p	000	
000		001		002
003		004		005
006		007		008

HD



Execução

```
#include <stdio.h>
int main() {
    int x;
    int* p;
    x = 10;
    p = &x;
    *p = *p + 7;
    printf("x = %d \n", x);
    printf("p = %p \n", p);
    printf("*p = %d \n", *p);
    printf("&x = %p \n", &x);
    printf("&p = %p \n", &p);
    return 0;
}
```

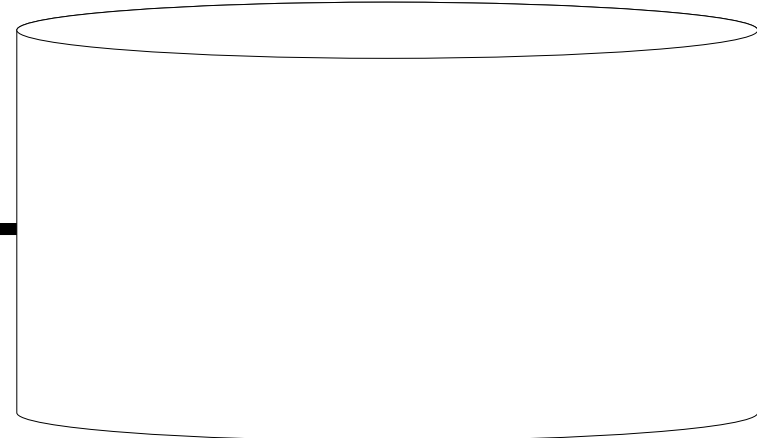
Monitor / Teclado

x = 17
p = 000
*p = 17

Memória

x	17	p	000	
000		001		002
003		004		005
006		007		008

HD



Execução

```
#include <stdio.h>
int main() {
    int x;
    int* p;
    x = 10;
    p = &x;
    *p = *p + 7;
    printf("x = %d \n", x);
    printf("p = %p \n", p);
    printf("*p = %d \n", *p);
    printf("&x = %p \n", &x);
    printf("&p = %p \n", &p);
    return 0;
}
```

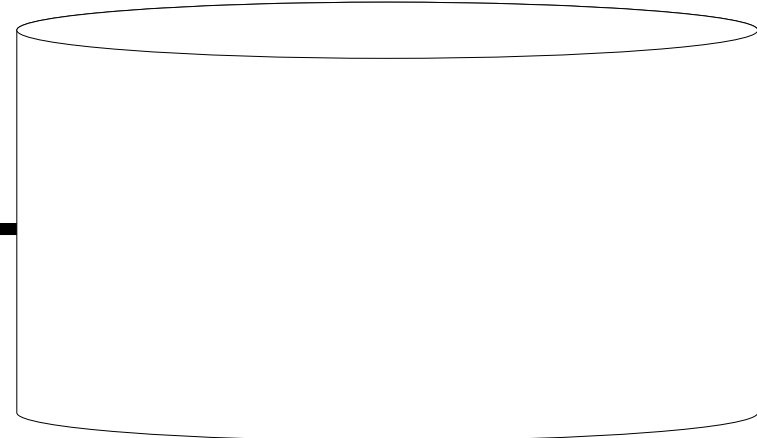
Monitor / Teclado

x = 17
p = 000
*p = 17
&x = 000

Memória

x	p	
17	000	
000	001	002
003	004	005
006	007	008

HD



Execução

```
#include <stdio.h>
int main() {
    int x;
    int* p;
    x = 10;
    p = &x;
    *p = *p + 7;
    printf("x = %d \n", x);
    printf("p = %p \n", p);
    printf("*p = %d \n", *p);
    printf("&x = %p \n", &x);
    printf("&p = %p \n", &p);
    return 0;
}
```

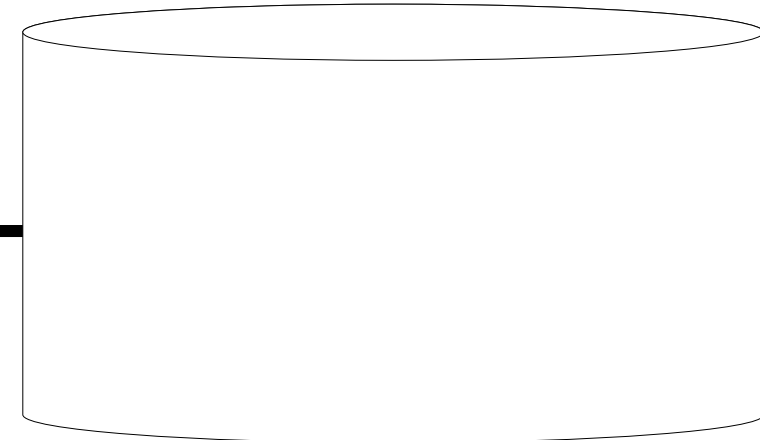
Monitor / Teclado

p = 000
*p = 17
&x = 000
&p = 001

Memória

x	p	
17	000	
000	001	002
003	004	005
006	007	008

HD



Execução

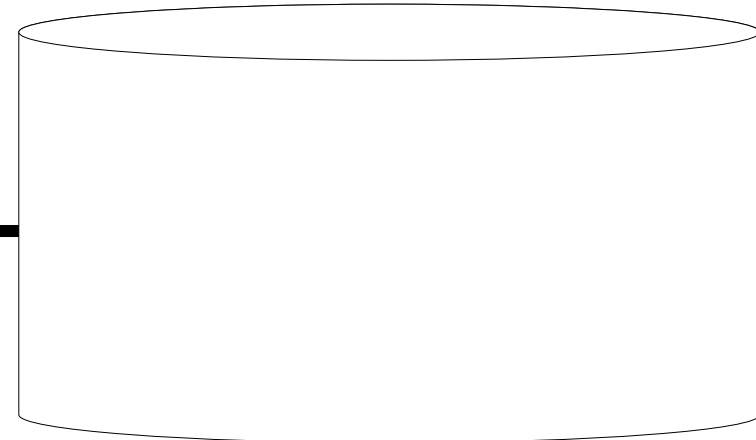
```
#include <stdio.h>
int main() {
    int x;
    int* p;
    x = 10;
    p = &x;
    *p = *p + 7;
    printf("x = %d \n", x);
    printf("p = %p \n", p);
    printf("*p = %d \n", *p);
    printf("&x = %p \n", &x);
    printf("&p = %p \n", &p);
    return 0;
}
```

Monitor / Teclado

Memória

000	001	002
003	004	005
006	007	008

HD



Ponteiros

Tem algum erro neste programa?

```
#include <stdio.h>
int main() {
    int x;
    int *p
    int *q;
    x = 10;
    p = &x;
    *p = *p + 10;
    *q = 30;
    return 0;;
}
```

Ponteiros

O que este programa imprime na tela?

```
#include <stdio.h>
void troca(int a, int b) {
    int aux;
    aux = a;
    a = b;
    b = aux;
}
int main() {
    int x, y;
    printf("Digite 2 numeros: ");
    scanf("%d %d", &x, &y);
    troca(x,y);
    printf("x = %d, y = %d",x, y);
    return 0;
}
```

Execução

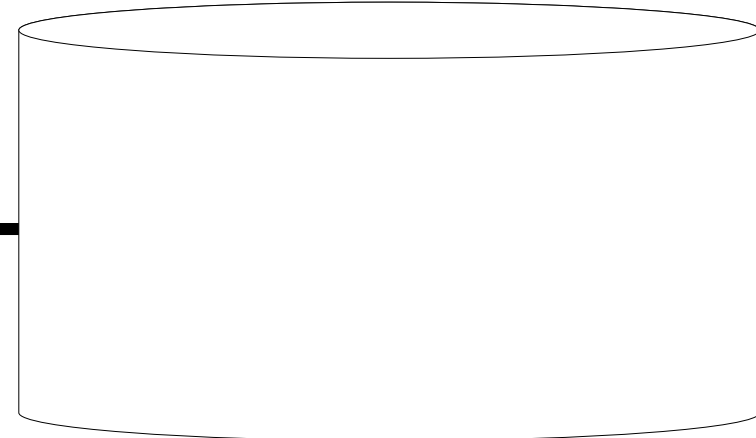
```
#include <stdio.h>
void troca(int a, int b) {
    int aux;
    aux = a;
    a = b;
    b = aux;
}
int main() {
    int x, y;
    printf("Digite 2 numeros: ");
    scanf("%d %d", &x, &y);
    troca(x,y);
    printf("x = %d, y = %d", x, y);
    return 0;
}
```

Monitor / Teclado

Memória

000	001	002
003	004	005
006	007	008

HD



Execução

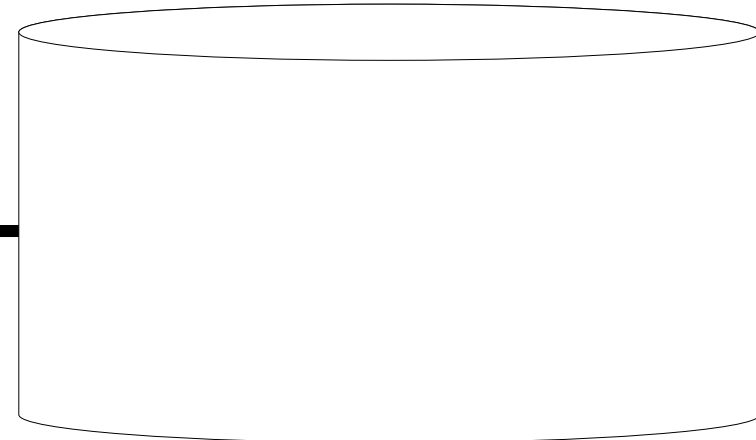
```
#include <stdio.h>
void troca(int a, int b) {
    int aux;
    aux = a;
    a = b;
    b = aux;
}
int main() {
    int x, y;
    printf("Digite 2 numeros: ");
    scanf("%d %d", &x, &y);
    troca(x,y);
    printf("x = %d, y = %d", x, y);
    return 0;
}
```

Monitor / Teclado

Memória

x	y	
000	001	002
003	004	005
006	007	008

HD



Execução

```
#include <stdio.h>
void troca(int a, int b) {
    int aux;
    aux = a;
    a = b;
    b = aux;
}
int main() {
    int x, y;
    printf("Digite 2 numeros: ");
    scanf("%d %d", &x, &y);
    troca(x,y);
    printf("x = %d, y = %d", x, y);
    return 0;
}
```

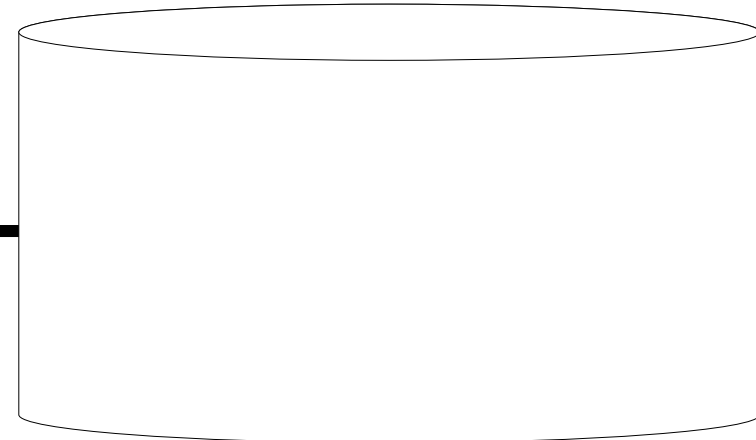
Monitor / Teclado

Digite 2 numeros:

Memória

x	y	
000	001	002
003	004	005
006	007	008

HD



Execução

```
#include <stdio.h>
void troca(int a, int b) {
    int aux;
    aux = a;
    a = b;
    b = aux;
}
int main() {
    int x, y;
    printf("Digite 2 numeros: ");
    scanf("%d %d", &x, &y);
    troca(x,y);
    printf("x = %d, y = %d", x, y);
    return 0;
}
```

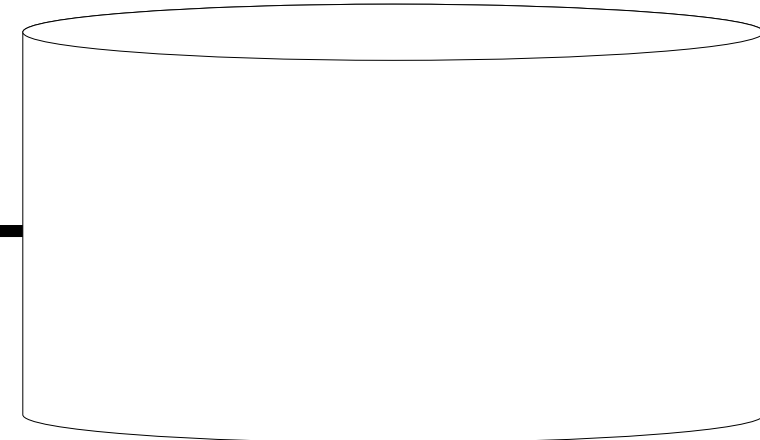
Monitor / Teclado

Digite 2 numeros:

Memória

x	y	
000	001	002
003	004	005
006	007	008

HD



Execução

```
#include <stdio.h>
void troca(int a, int b) {
    int aux;
    aux = a;
    a = b;
    b = aux;
}
int main() {
    int x, y;
    printf("Digite 2 numeros: ");
    scanf("%d %d", &x, &y);
    troca(x,y);
    printf("x = %d, y = %d", x, y);
    return 0;
}
```

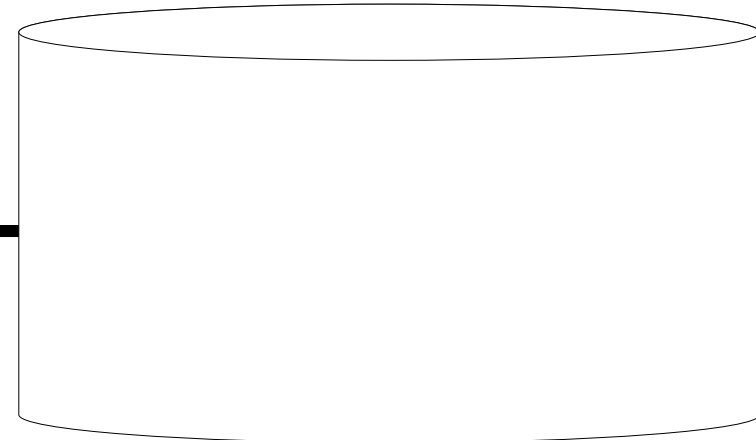
Monitor / Teclado

Digite 2 numeros: 3 8

Memória

x	y	
000	001	002
003	004	005
006	007	008

HD



Execução

```
#include <stdio.h>
void troca(int a, int b) {
    int aux;
    aux = a;
    a = b;
    b = aux;
}
int main() {
    int x, y;
    printf("Digite 2 numeros: ");
    scanf("%d %d", &x, &y);
    troca(x,y);
    printf("x = %d, y = %d", x, y);
    return 0;
}
```

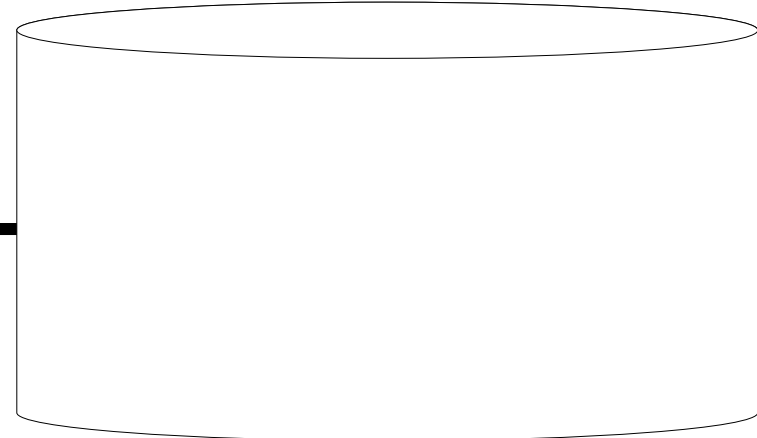
Monitor / Teclado

Digite 2 numeros: 3 8 <ENTER>

Memória

x	y	
3	8	
000	001	002
003	004	005
006	007	008

HD



Execução

```
#include <stdio.h>
void troca(int a, int b) {
    int aux;
    aux = a;
    a = b;
    b = aux;
}
int main() {
    int x, y;
    printf("Digite 2 numeros: ");
    scanf("%d %d", &x, &y);
    troca(x, y);
    printf("x = %d, y = %d", x, y);
    return 0;
}
```

*O computador chama
troca() para trabalhar.*

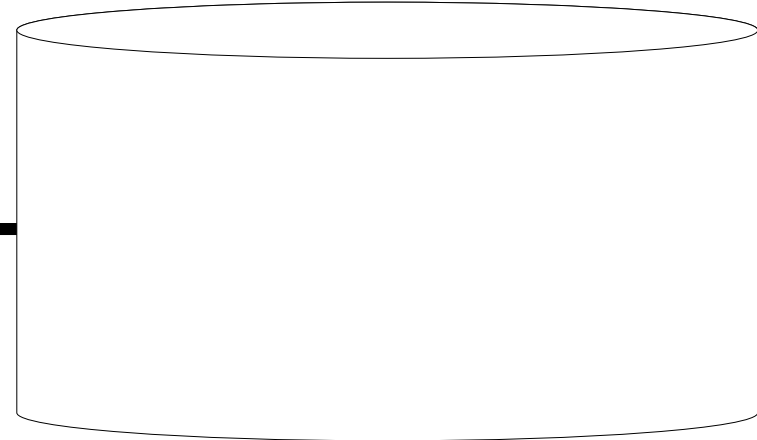
Monitor / Teclado

Digite 2 numeros: 3 8 <ENTER>

Memória

x	y	
3	8	
000	001	002
003	004	005
006	007	008

HD



Execução

```
#include <stdio.h>
void troca(int a, int b) {
    int aux;
    aux = a;
    a = b;
    b = aux;
}
int main() {
    int x, y;
    printf("Digite 2 numeros: ");
    scanf("%d %d", &x, &y);
    troca(x,y);
    printf("x = %d, y = %d", x, y);
    return 0;
}
```

*O main() fica parado
esperando troca()
finalizar.*

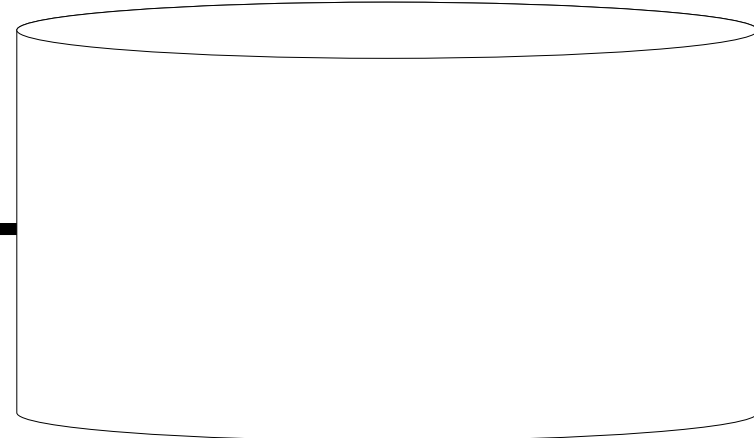
Monitor / Teclado

Digite 2 numeros: 3 8 <ENTER>

Memória

x	y	
3	8	
000	001	002
003	004	005
006	007	008

HD



Execução

```
#include <stdio.h>
void troca(int a, int b) {
    int aux;
    aux = a;
    a = b;
    b = aux;
}
int main() {
    int x, y;
    printf("Digite 2 numeros: ");
    scanf("%d %d", &x, &y);
    troca(x,y);
    printf("x = %d, y = %d", x, y);
    return 0;
}
```

As variáveis a e b são criadas na memória. Estas variáveis são exclusivas da função troca() e ocupa outro espaço na memória diferente de x e y da função main(). Para não confundirmos, chamaremos a e b da função troca() de $a_{troca()}$ e $b_{troca()}$

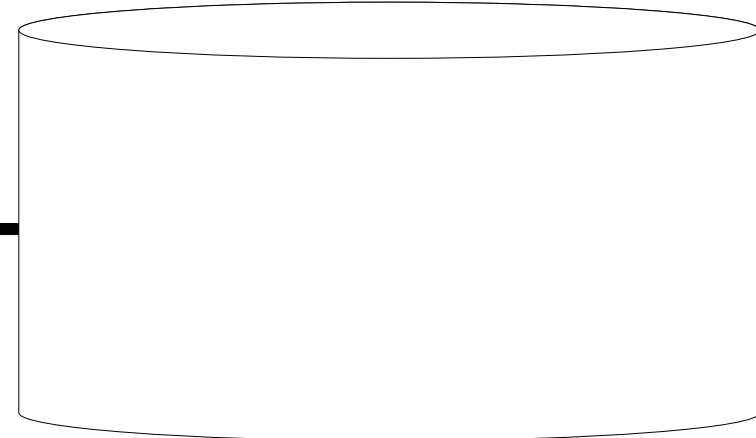
Monitor / Teclado

Digite 2 numeros: 3 8 <ENTER>

Memória

x	y	
3	8	
000	001	002
$a_{troca()}$	$b_{troca()}$	
3	8	
003	004	005
006	007	008

HD



Execução

```
#include <stdio.h>
void troca(int a, int b) {
    int aux;
    aux = a;
    a = b;
    b = aux;
}
int main() {
    int x, y;
    printf("Digite 2 numeros: ");
    scanf("%d %d", &x, &y);
    troca(x,y);
    printf("x = %d, y = %d", x, y);
    return 0;
}
```

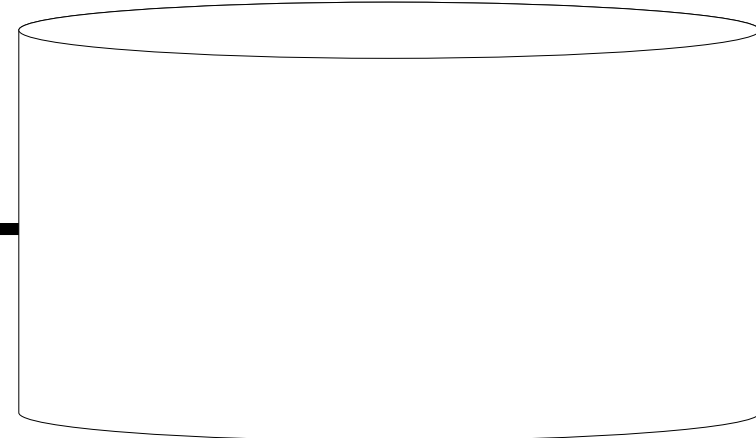
Monitor / Teclado

Digite 2 numeros: 3 8 <ENTER>

Memória

x	y	
3	8	
000	001	002
a _{troca()}	b _{troca()}	aux _{troca()}
3	8	
003	004	005
006	007	008

HD



Execução

```
#include <stdio.h>
void troca(int a, int b) {
    int aux;
    aux = a;
    a = b;
    b = aux;
}
int main() {
    int x, y;
    printf("Digite 2 numeros: ");
    scanf("%d %d", &x, &y);
    troca(x,y);
    printf("x = %d, y = %d", x, y);
    return 0;
}
```

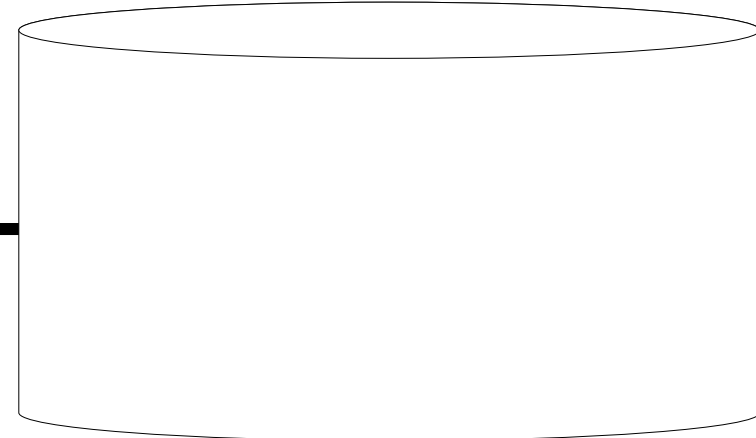
Monitor / Teclado

Digite 2 numeros: 3 8 <ENTER>

Memória

x	y	
3	8	
000	001	002
a _{troca()}	b _{troca()}	aux _{troca()}
3	8	3
003	004	005
006	007	008

HD



Execução

```
#include <stdio.h>
void troca(int a, int b) {
    int aux;
    aux = a;
    a = b;
    b = aux;
}
int main() {
    int x, y;
    printf("Digite 2 numeros: ");
    scanf("%d %d", &x, &y);
    troca(x,y);
    printf("x = %d, y = %d", x, y);
    return 0;
}
```

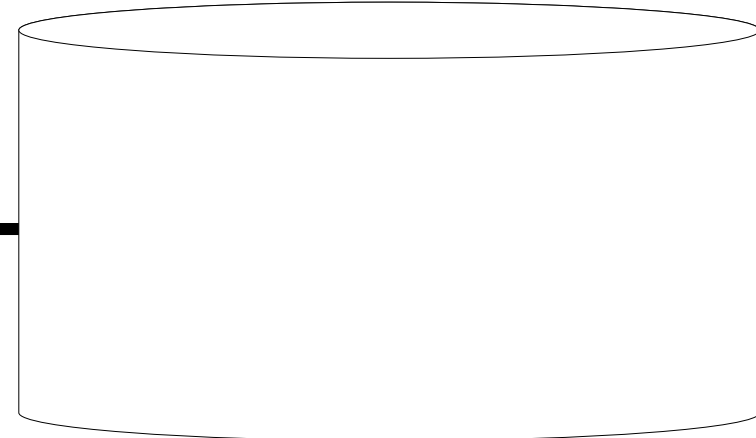
Monitor / Teclado

Digite 2 numeros: 3 8 <ENTER>

Memória

x	y	
3	8	
000	001	002
a _{troca()}	b _{troca()}	aux _{troca()}
8	8	3
003	004	005
006	007	008

HD



Execução

```
#include <stdio.h>
void troca(int a, int b) {
    int aux;
    aux = a;
    a = b;
    b = aux;
}
int main() {
    int x, y;
    printf("Digite 2 numeros: ");
    scanf("%d %d", &x, &y);
    troca(x,y);
    printf("x = %d, y = %d", x, y);
    return 0;
}
```

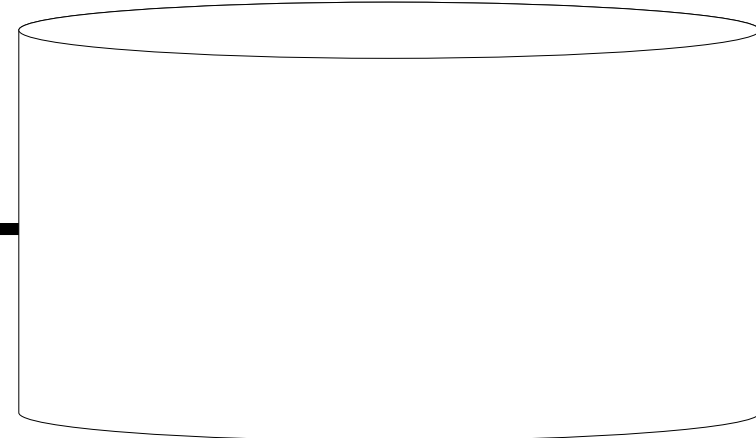
Monitor / Teclado

Digite 2 numeros: 3 8 <ENTER>

Memória

x	y	
3	8	
000	001	002
a _{troca()}	b _{troca()}	aux _{troca()}
8	3	3
003	004	005
006	007	008

HD



Execução

```
#include <stdio.h>
void troca(int a, int b) {
    int aux;
    aux = a;
    a = b;
    b = aux;
}
int main() {
    int x, y;
    printf("Digite 2 numeros: ");
    scanf("%d %d", &x, &y);
    troca(x,y);
    printf("x = %d, y = %d", x, y);
    return 0;
}
```

Como a função troca() não retorna nada, as variáveis são perdidas.

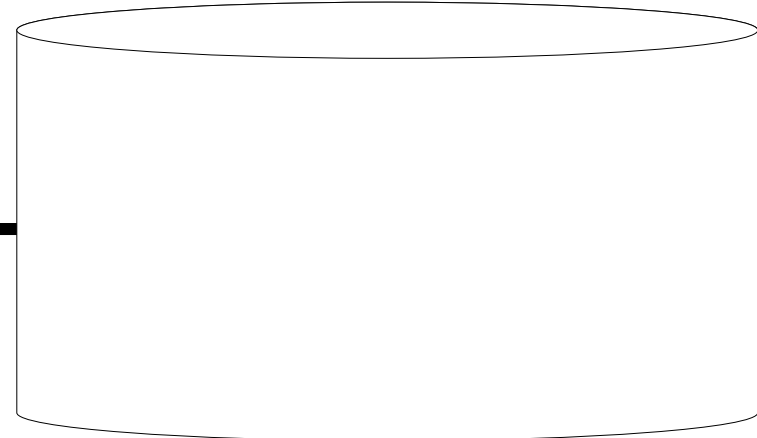
Monitor / Teclado

Digite 2 numeros: 3 8 <ENTER>

Memória

x	y	
3	8	
000	001	002
003	004	005
006	007	008

HD



Execução

```
#include <stdio.h>
void troca(int a, int b) {
    int aux;
    aux = a;
    a = b;
    b = aux;
}
int main() {
    int x, y;
    printf("Digite 2 numeros: ");
    scanf("%d %d", &x, &y);
    troca(x,y);
    printf("x = %d, y = %d", x, y);
    return 0;
}
```

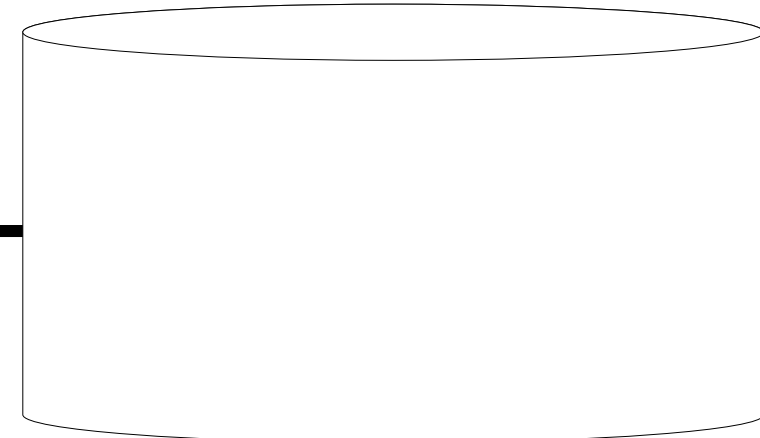
Monitor / Teclado

Digite 2 numeros: 3 8 <ENTER>
X = 3, y = 8

Memória

x	y	
3	8	
000	001	002
003	004	005
006	007	008

HD



Execução

```
#include <stdio.h>
void troca(int a, int b) {
    int aux;
    aux = a;
    a = b;
    b = aux;
}
int main() {
    int x, y;
    printf("Digite 2 numeros: ");
    scanf("%d %d", &x, &y);
    troca(x,y);
    printf("x = %d, y = %d", x, y);
    return 0;
}
```

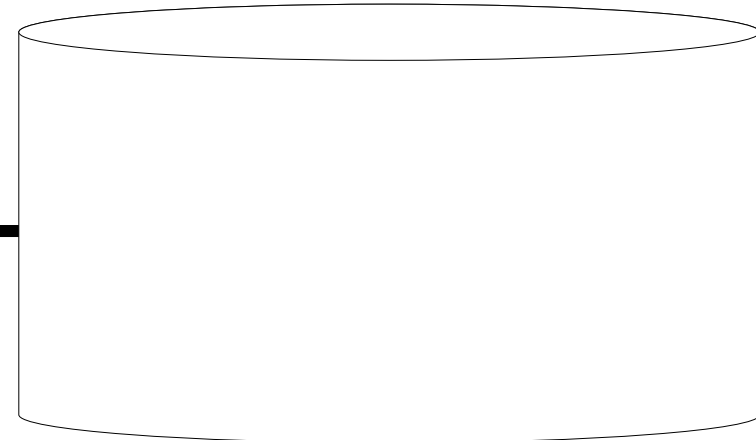


Monitor / Teclado

Memória

000	001	002
003	004	005
006	007	008

HD



Ponteiros

E agora? O que este programa imprime na tela?

```
#include <stdio.h>
void troca(int *a, int *b) {
    int aux;
    aux = *a;
    *a = *b;
    *b = aux;
}
int main() {
    int x, y;
    printf("Digite 2 numeros: ");
    scanf("%d %d", &x, &y);
    troca(&x, &y);
    printf("x = %d, y = %d", x, y);
    return 0;
}
```

Execução

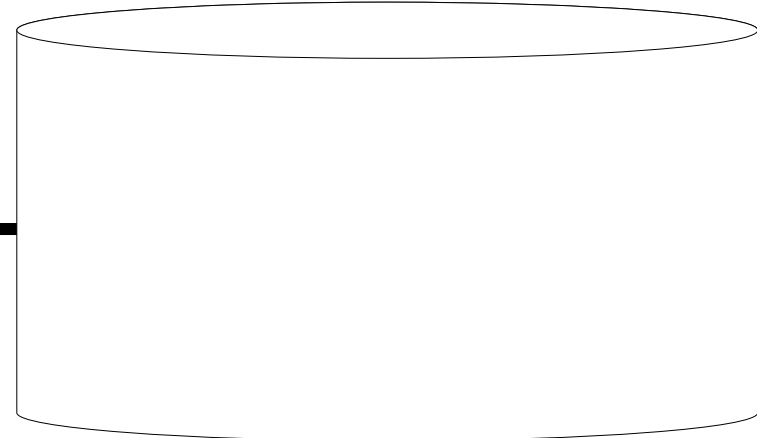
```
#include <stdio.h>
void troca(int *a, int *b) {
    int aux;
    aux = *a;
    *a = *b;
    *b = aux;
}
int main() {
    int x, y;
    printf("Digite 2 numeros: ");
    scanf("%d %d", &x, &y);
    troca(&x, &y);
    printf("x = %d, y = %d", x, y);
    return 0;
}
```

Monitor / Teclado

Memória

000	001	002
003	004	005
006	007	008

HD



Execução

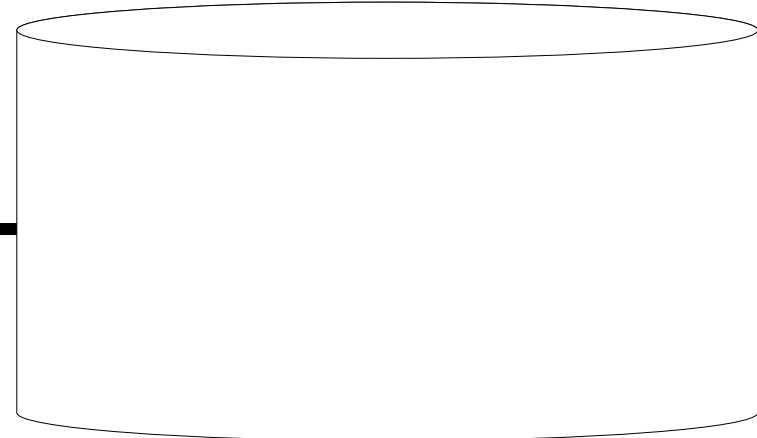
```
#include <stdio.h>
void troca(int *a, int *b) {
    int aux;
    aux = *a;
    *a = *b;
    *b = aux;
}
int main() {
    int x, y;
    printf("Digite 2 numeros: ");
    scanf("%d %d", &x, &y);
    troca(&x, &y);
    printf("x = %d, y = %d", x, y);
    return 0;
}
```

Monitor / Teclado

Memória

x	y	
000	001	002
003	004	005
006	007	008

HD



Execução

```
#include <stdio.h>
void troca(int *a, int *b) {
    int aux;
    aux = *a;
    *a = *b;
    *b = aux;
}
int main() {
    int x, y;
    printf("Digite 2 numeros: ");
    scanf("%d %d", &x, &y);
    troca(&x, &y);
    printf("x = %d, y = %d", x, y);
    return 0;
}
```

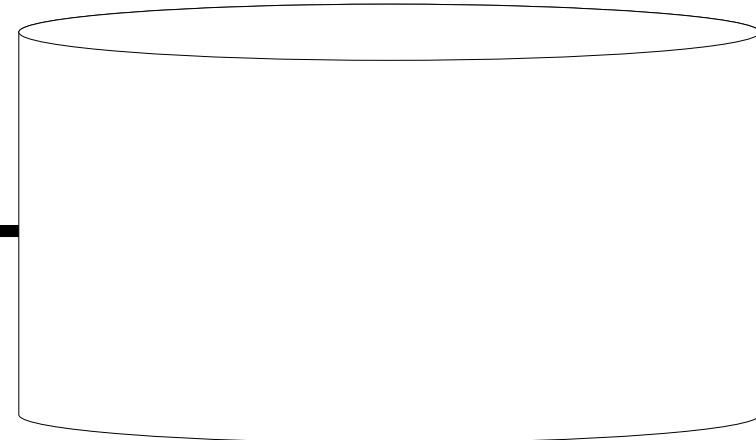
Monitor / Teclado

Digite 2 numeros:

Memória

x	y	
000	001	002
003	004	005
006	007	008

HD



Execução

```
#include <stdio.h>
void troca(int *a, int *b) {
    int aux;
    aux = *a;
    *a = *b;
    *b = aux;
}
int main() {
    int x, y;
    printf("Digite 2 numeros: ");
    scanf("%d %d", &x, &y);
    troca(&x, &y);
    printf("x = %d, y = %d", x, y);
    return 0;
}
```

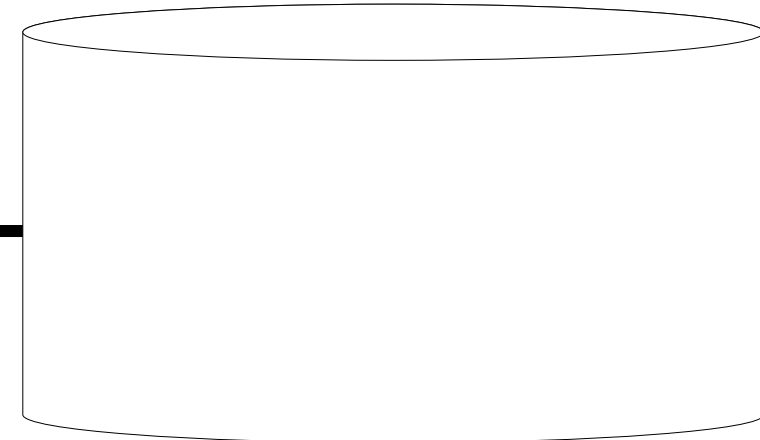
Monitor / Teclado

Digite 2 numeros:

Memória

x	y	
000	001	002
003	004	005
006	007	008

HD



Execução

```
#include <stdio.h>
void troca(int *a, int *b) {
    int aux;
    aux = *a;
    *a = *b;
    *b = aux;
}
int main() {
    int x, y;
    printf("Digite 2 numeros: ");
    scanf("%d %d", &x, &y);
    troca(&x, &y);
    printf("x = %d, y = %d", x, y);
    return 0;
}
```

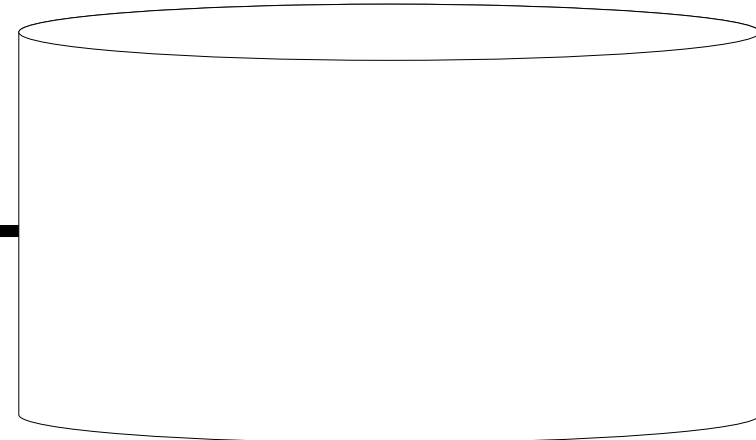
Monitor / Teclado

Digite 2 numeros: 3 8

Memória

x	y	
000	001	002
003	004	005
006	007	008

HD



Execução

```
#include <stdio.h>
void troca(int *a, int *b) {
    int aux;
    aux = *a;
    *a = *b;
    *b = aux;
}
int main() {
    int x, y;
    printf("Digite 2 numeros: ");
    scanf("%d %d", &x, &y);
    troca(&x, &y);
    printf("x = %d, y = %d", x, y);
    return 0;
}
```

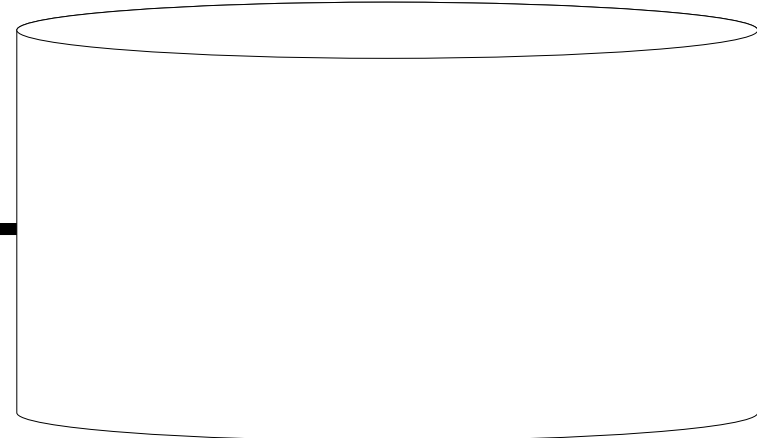
Monitor / Teclado

Digite 2 numeros: 3 8 <ENTER>

Memória

x	y	
3	8	
000	001	002
003	004	005
006	007	008

HD



Execução

```
#include <stdio.h>
void troca(int *a, int *b) {
    int aux;
    aux = *a;
    *a = *b;
    *b = aux;
}
int main() {
    int x, y;
    printf("Digite 2 numeros: ");
    scanf("%d %d", &x, &y);
    troca(&x, &y);
    printf("x = %d, y = %d", x, y);
    return 0;
}
```

*O computador chama
troca() para trabalhar.*

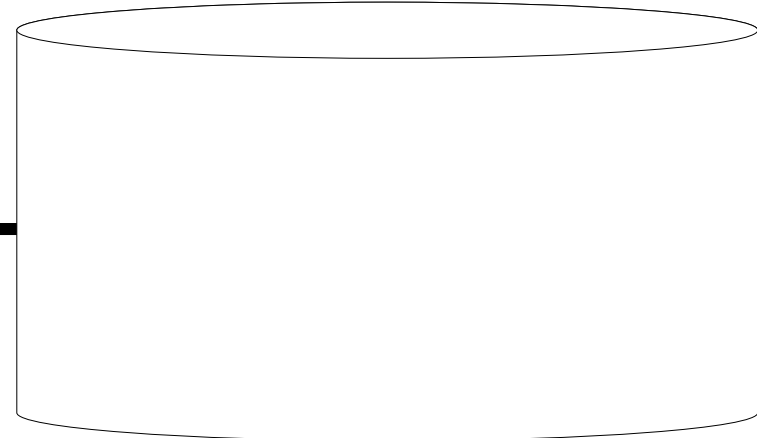
Monitor / Teclado

Digite 2 numeros: 3 8 <ENTER>

Memória

x	y	
3	8	
000	001	002
003	004	005
006	007	008

HD



Execução

```
#include <stdio.h>
void troca(int *a, int *b) {
    int aux;
    aux = *a;
    *a = *b;
    *b = aux;
}
int main() {
    int x, y;
    printf("Digite 2 numeros: ");
    scanf("%d %d", &x, &y);
    troca(&x, &y);
    printf("x = %d, y = %d", x, y);
    return 0;
}
```

*O main() fica parado
esperando troca()
finalizar.*

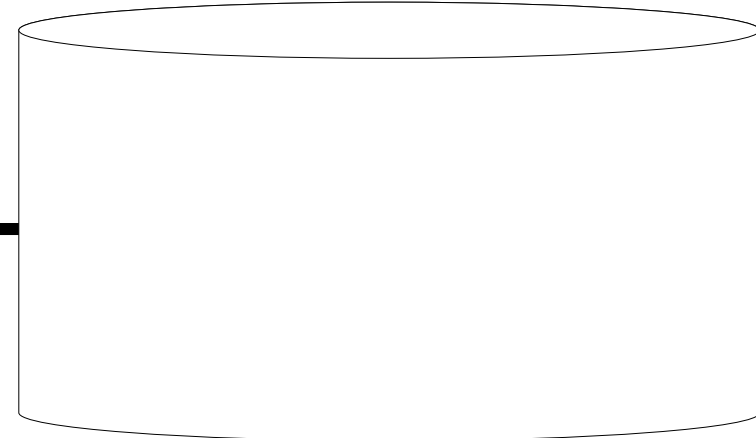
Monitor / Teclado

Digite 2 numeros: 3 8 <ENTER>

Memória

x	y	
3	8	
000	001	002
003	004	005
006	007	008

HD



Execução

```
#include <stdio.h>
void troca(int *a, int *b) {
    int aux;
    aux = *a;
    *a = *b;
    *b = aux;
}
int main() {
    int x, y;
    printf("Digite 2 numeros: ");
    scanf("%d %d", &x, &y);
    troca(&x, &y);
    printf("x = %d, y = %d", x, y);
    return 0;
}
```

As variáveis a e b são criadas na memória. Estas variáveis são exclusivas da função troca() e ocupa outro espaço na memória diferente de x e y da função main(). Para não confundirmos, chamaremos a e b da função troca() de $a_{troca()}$ e $b_{troca()}$

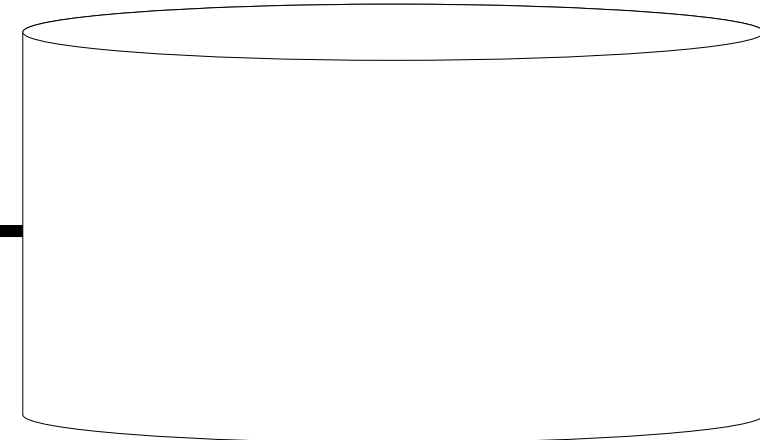
Monitor / Teclado

Digite 2 numeros: 3 8 <ENTER>

Memória

x	3	y	8	
000		001		002
$a_{troca()}$		$b_{troca()}$		
003	000	004	001	005
006		007		008

HD



Execução

```
#include <stdio.h>
void troca(int *a, int *b) {
    int aux;
    aux = *a;
    *a = *b;
    *b = aux;
}
int main() {
    int x, y;
    printf("Digite 2 numeros: ");
    scanf("%d %d", &x, &y);
    troca(&x, &y);
    printf("x = %d, y = %d", x, y);
    return 0;
}
```

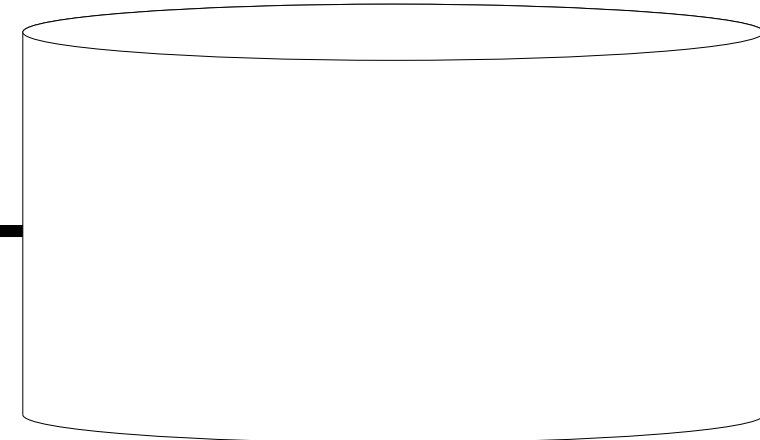
Monitor / Teclado

Digite 2 numeros: 3 8 <ENTER>

Memória

x	y	
3	8	
000	001	002
a _{troca()}	b _{troca()}	aux _{troca()}
003 000	004 001	005
006	007	008

HD



Execução

```
#include <stdio.h>
void troca(int *a, int *b) {
    int aux;
    aux = *a;
    *a = *b;
    *b = aux;
}
int main() {
    int x, y;
    printf("Digite 2 numeros: ");
    scanf("%d %d", &x, &y);
    troca(&x, &y);
    printf("x = %d, y = %d", x, y);
    return 0;
}
```

1. O computador vai em $a_{troca()}$
2. Pega o seu conteúdo (000)
3. Vai neste endereço 000
4. Retorna o conteúdo de 000, que é o valor 3

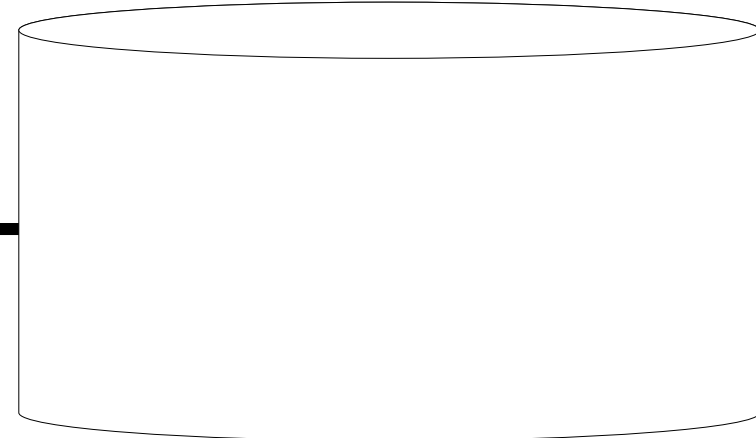
Monitor / Teclado

Digite 2 numeros: 3 8 <ENTER>

Memória

x	y	
3	8	
000	001	002
$a_{troca()}$	$b_{troca()}$	$aux_{troca()}$
003 000	004 001	005
006	007	008

HD



Execução

```
#include <stdio.h>
void troca(int *a, int *b) {
    int aux;
    aux = *a;
    *a = *b;
    *b = aux;
}
int main() {
    int x, y;
    printf("Digite 2 numeros: ");
    scanf("%d %d", &x, &y);
    troca(&x, &y);
    printf("x = %d, y = %d", x, y);
    return 0;
}
```

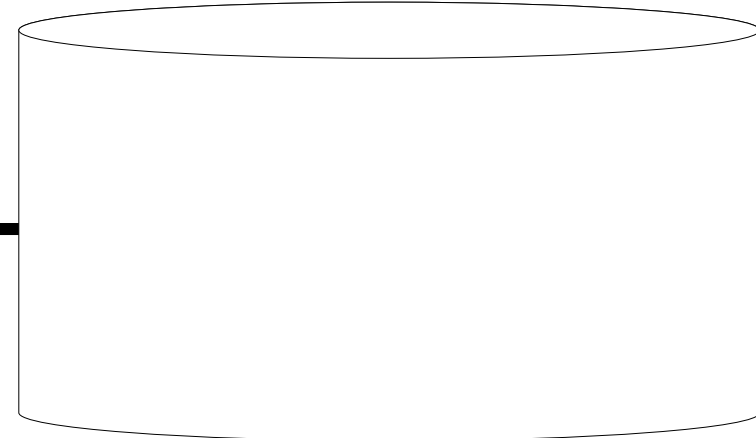
Monitor / Teclado

Digite 2 numeros: 3 8 <ENTER>

Memória

x	y	
3	8	
000	001	002
a _{troca()}	b _{troca()}	aux _{troca()}
003 000	004 001	005 3
006	007	008

HD



Execução

```
#include <stdio.h>
void troca(int *a, int *b) {
    int aux;
    aux = *a;
    *a = *b;
    *b = aux;
}
int main() {
    int x, y;
    printf("Digite 2 numeros: ");
    scanf("%d %d", &x, &y);
    troca(&x, &y);
    printf("x = %d, y = %d", x, y);
    return 0;
}
```

1. O computador vai em $b_{troca()}$
2. Pega o seu conteúdo (001)
3. Vai neste endereço 001
4. Retorna o conteúdo de 001, que é o valor 8

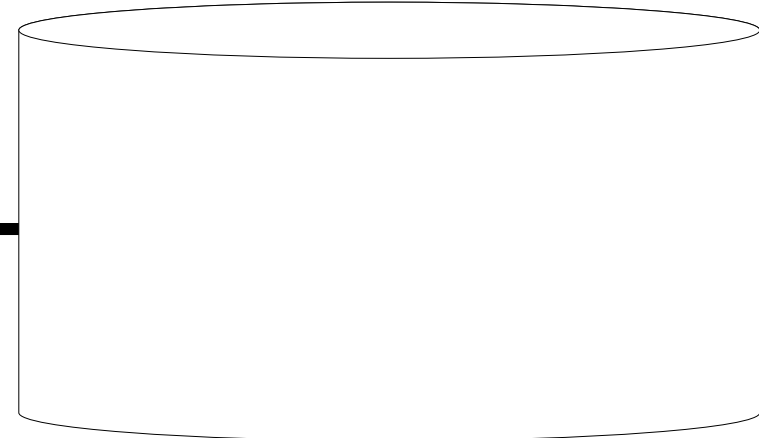
Monitor / Teclado

Digite 2 numeros: 3 8 <ENTER>

Memória

x	y	
3	8	
000	001	002
a _{troca()}	b _{troca()}	aux _{troca()}
003 000	004 001	005 3
006	007	008

HD



Execução

```
#include <stdio.h>
void troca(int *a, int *b) {
    int aux;
    aux = *a;
    *a = *b;
    *b = aux;
}
int main() {
    int x, y;
    printf("Digite 2 numeros: ");
    scanf("%d %d", &x, &y);
    troca(&x, &y);
    printf("x = %d, y = %d", x, y);
    return 0;
}
```

1. O computador vai em $a_{troca()}$
2. Pega o seu conteúdo (000)
3. Vai neste endereço 000
4. Salva o $*b$ no conteúdo de 000, que é o valor 8

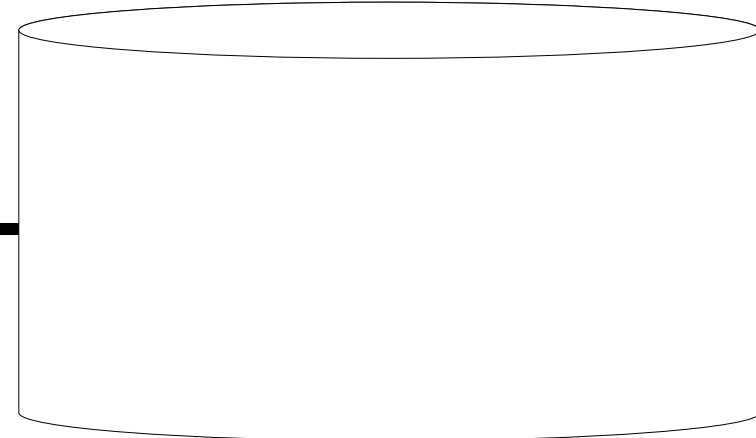
Monitor / Teclado

Digite 2 numeros: 3 8 <ENTER>

Memória

x	y	
8	8	
000	001	002
$a_{troca()}$	$b_{troca()}$	$aux_{troca()}$
003 000	004 001	005 3
006	007	008

HD



Execução

```
#include <stdio.h>
void troca(int *a, int *b) {
    int aux;
    aux = *a;
    *a = *b;
    *b = aux;
}
int main() {
    int x, y;
    printf("Digite 2 numeros: ");
    scanf("%d %d", &x, &y);
    troca(&x, &y);
    printf("x = %d, y = %d", x, y);
    return 0;
}
```

1. O computador vai em $b_{troca()}$
2. Pega o seu conteúdo (001)
3. Vai neste endereço 001
4. Salva o valor de aux, que é 3, no conteúdo de 001.

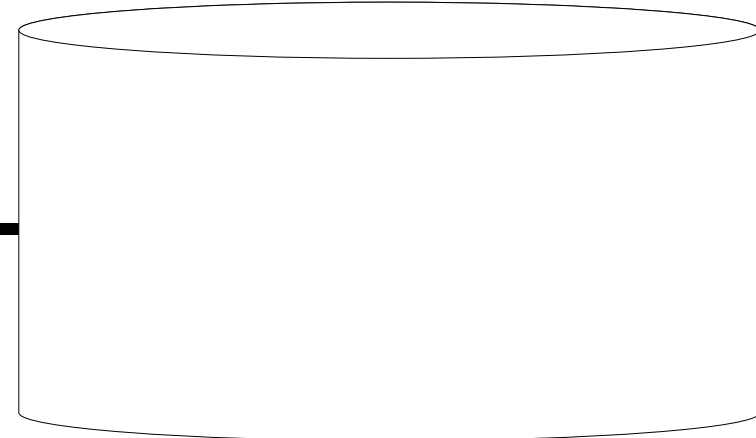
Monitor / Teclado

Digite 2 numeros: 3 8 <ENTER>

Memória

x	y	
8	3	
000	001	002
a _{troca()}	b _{troca()}	aux _{troca()}
003 000	004 001	005 3
006	007	008

HD



Execução

```
#include <stdio.h>
void troca(int *a, int *b) {
    int aux;
    aux = *a;
    *a = *b;
    *b = aux;
}
int main() {
    int x, y;
    printf("Digite 2 numeros: ");
    scanf("%d %d", &x, &y);
    troca(&x, &y);
    printf("x = %d, y = %d", x, y);
    return 0;
}
```

Como a função troca() não retorna nada, as variáveis são perdidas. Porém, note que o uso de ponteiros permitiu troca() manipular x e y do main().

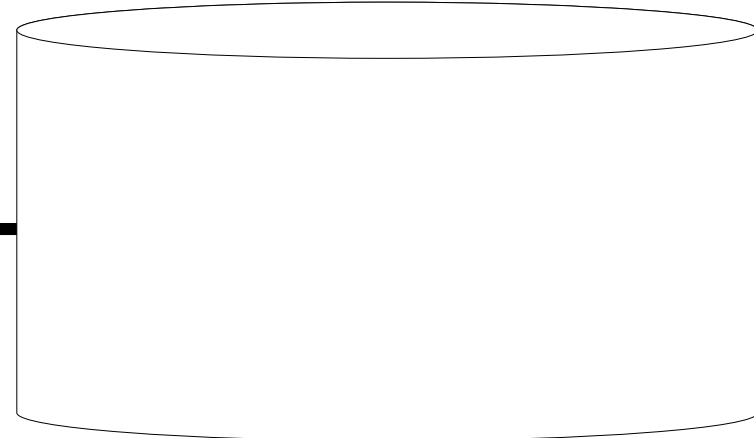
Monitor / Teclado

Digite 2 numeros: 3 8 <ENTER>

Memória

x	y	
8	3	
000	001	002
003	004	005
006	007	008

HD



Execução

```
#include <stdio.h>
void troca(int *a, int *b) {
    int aux;
    aux = *a;
    *a = *b;
    *b = aux;
}
int main() {
    int x, y;
    printf("Digite 2 numeros: ");
    scanf("%d %d", &x, &y);
    troca(&x, &y);
    printf("x = %d, y = %d", x, y);
    return 0;
}
```

Como a função troca() não retorna nada, as variáveis são perdidas. Porém, note que o uso de ponteiros permitiu troca() manipular x e y do main().

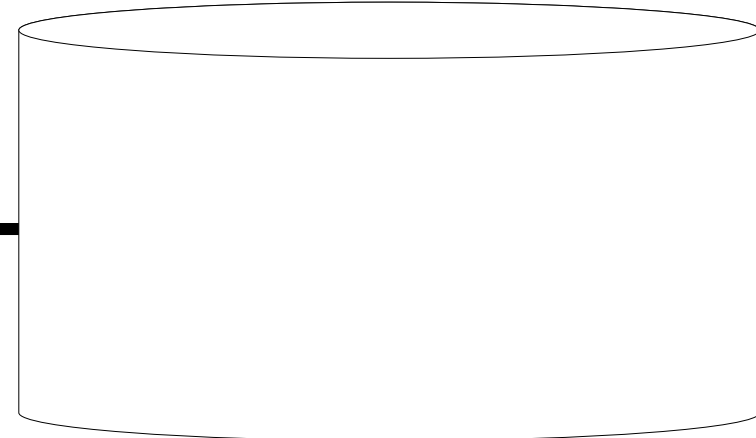
Monitor / Teclado

Digite 2 numeros: 3 8 <ENTER>

Memória

x	y	
8	3	
000	001	002
003	004	005
006	007	008

HD



Execução

```
#include <stdio.h>
void troca(int *a, int *b) {
    int aux;
    aux = *a;
    *a = *b;
    *b = aux;
}
int main() {
    int x, y;
    printf("Digite 2 numeros: ");
    scanf("%d %d", &x, &y);
    troca(&x, &y);
    printf("x = %d, y = %d", x, y);
    return 0;
}
```

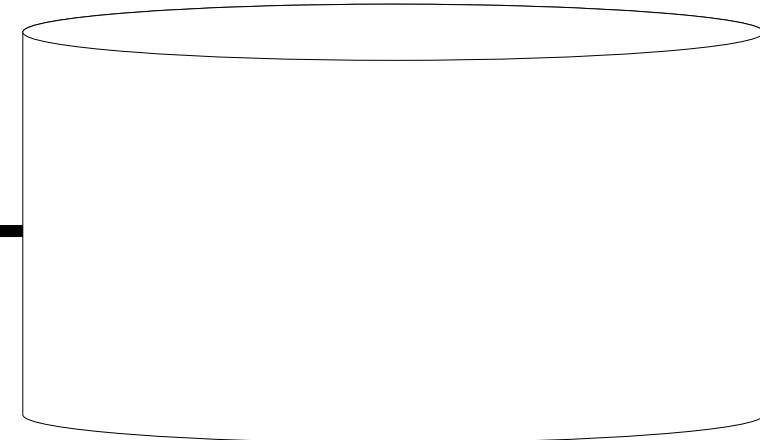
Monitor / Teclado

Digite 2 numeros: 3 8 <ENTER>
x = 8, y = 3

Memória

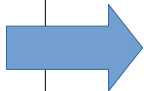
x	8	y	3	
000		001		002
003		004		005
006		007		008

HD



Execução

```
#include <stdio.h>
void troca(int *a, int *b) {
    int aux;
    aux = *a;
    *a = *b;
    *b = aux;
}
int main() {
    int x, y;
    printf("Digite 2 numeros: ");
    scanf("%d %d", &x, &y);
    troca(&x, &y);
    printf("x = %d, y = %d", x, y);
    return 0;
}
```

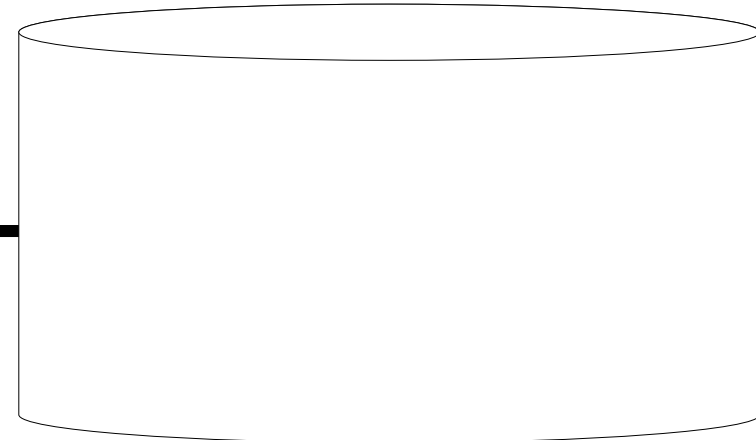


Monitor / Teclado

Memória

000	001	002
003	004	005
006	007	008

HD



Ponteiros

Se forem digitados 5 e 8, o que aparece na tela?

```
#include <stdio.h>
void operacoes(int a,int b,int* res1,int* res2) {
    *res1 = a + b;
    *res2 = a * b;
}
int main() {
    int x, y, soma, prod;
    printf("Digite x e y: ");
    scanf("%d %d", &x, &y);
    operacoes(x, y, &soma , &prod);
    printf("soma = %d, prod = %d",soma,prod);
    return 0;
}
```

Execução

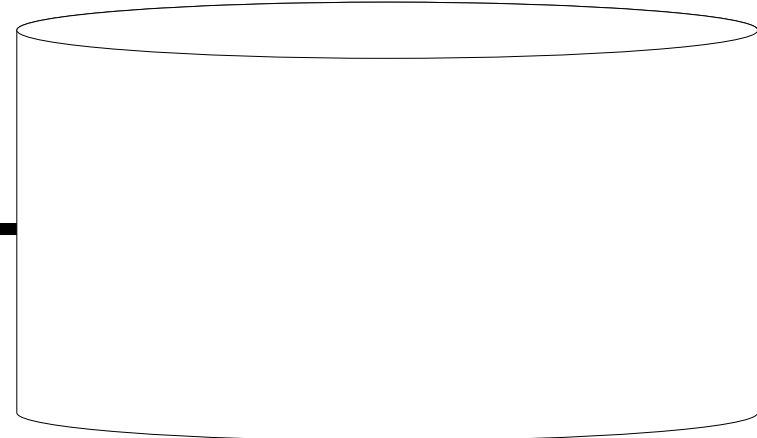
```
#include <stdio.h>
void operacoes(int a,int b,int*
res1,int* res2){
    *res1 = a + b;
    *res2 = a * b;
}
int main() {
    int x, y, soma, prod;
    printf("Digite x e y: ");
    scanf("%d %d", &x, &y);
    operacoes(x, y, &soma , &prod);
    printf("soma = %d, prod =
%d", soma, prod);
    return 0;
}
```

Monitor / Teclado

Memória

000	001	002
003	004	005
006	007	008

HD



Execução

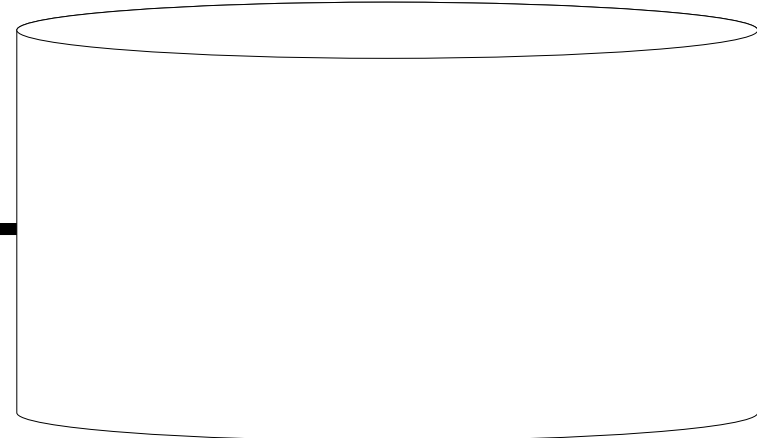
```
#include <stdio.h>
void operacoes(int a,int b,int*
res1,int* res2){
    *res1 = a + b;
    *res2 = a * b;
}
int main() {
    int x, y, soma, prod;
    printf("Digite x e y: ");
    scanf("%d %d", &x, &y);
    operacoes(x, y, &soma , &prod);
    printf("soma = %d, prod =
%d",soma,prod);
    return 0;
}
```

Monitor / Teclado

Memória

x	y	soma
000	001	002
prod		
003	004	005
006	007	008

HD



Execução

```
#include <stdio.h>
void operacoes(int a,int b,int*
res1,int* res2){
    *res1 = a + b;
    *res2 = a * b;
}
int main() {
    int x, y, soma, prod;
    printf("Digite x e y: ");
    scanf("%d %d", &x, &y);
    operacoes(x, y, &soma , &prod);
    printf("soma = %d, prod =
%d", soma, prod);
    return 0;
}
```

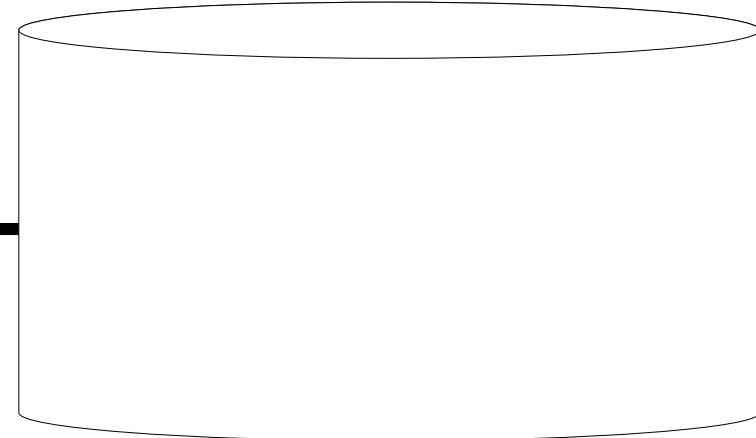
Monitor / Teclado

Digite x e y:

Memória

x	y	soma
000	001	002
prod		
003	004	005
006	007	008

HD



Execução

```
#include <stdio.h>
void operacoes(int a,int b,int*
res1,int* res2){
    *res1 = a + b;
    *res2 = a * b;
}
int main() {
    int x, y, soma, prod;
    printf("Digite x e y: ");
    scanf("%d %d", &x, &y);
    operacoes(x, y, &soma , &prod);
    printf("soma = %d, prod =
%d", soma, prod);
    return 0;
}
```

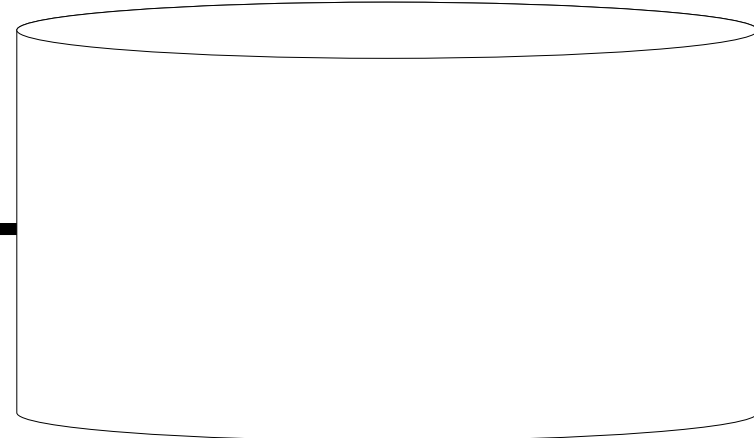
Monitor / Teclado

Digite x e y:

Memória

x	y	soma
000	001	002
prod		
003	004	005
006	007	008

HD



Execução

```
#include <stdio.h>
void operacoes(int a,int b,int*
res1,int* res2){
    *res1 = a + b;
    *res2 = a * b;
}
int main() {
    int x, y, soma, prod;
    printf("Digite x e y: ");
    scanf("%d %d", &x, &y);
    operacoes(x, y, &soma , &prod);
    printf("soma = %d, prod =
%d", soma, prod);
    return 0;
}
```

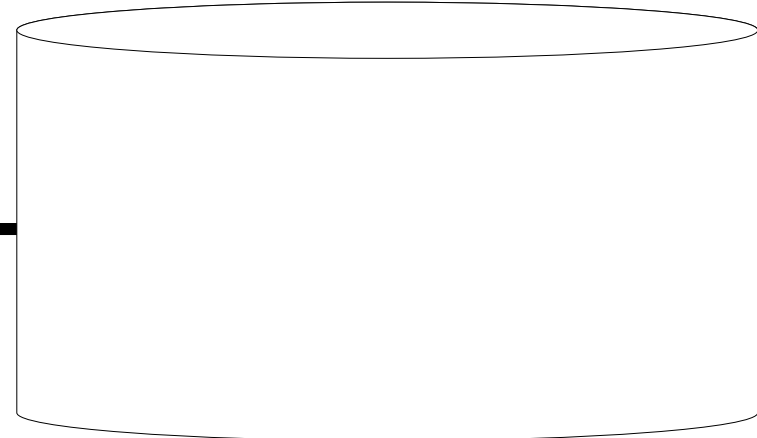
Monitor / Teclado

Digite x e y: 5 8

Memória

x	y	soma
000	001	002
prod		
003	004	005
006	007	008

HD



Execução

```
#include <stdio.h>
void operacoes(int a,int b,int*
res1,int* res2){
    *res1 = a + b;
    *res2 = a * b;
}
int main() {
    int x, y, soma, prod;
    printf("Digite x e y: ");
    scanf("%d %d", &x, &y);
    operacoes(x, y, &soma , &prod);
    printf("soma = %d, prod =
%d", soma, prod);
    return 0;
}
```

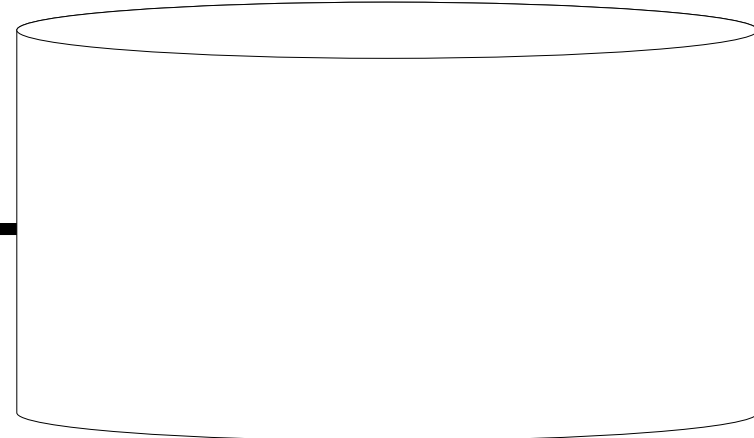
Monitor / Teclado

Digite x e y: 5 8 <ENTER>

Memória

x	y	soma
5	8	
000	001	002
prod		
003	004	005
006	007	008

HD



Execução

```
#include <stdio.h>
void operacoes(int a,int b,int*
res1,int* res2){
    *res1 = a + b;
    *res2 = a * b;
}
int main() {
    int x, y, soma, prod;
    printf("Digite x e y: ");
    scanf("%d %d", &x, &y);
    operacoes(x, y, &soma , &prod);
    printf("soma = %d, prod =
%d", soma, prod);
    return 0;
}
```

O computador chama operacoes() para trabalhar.

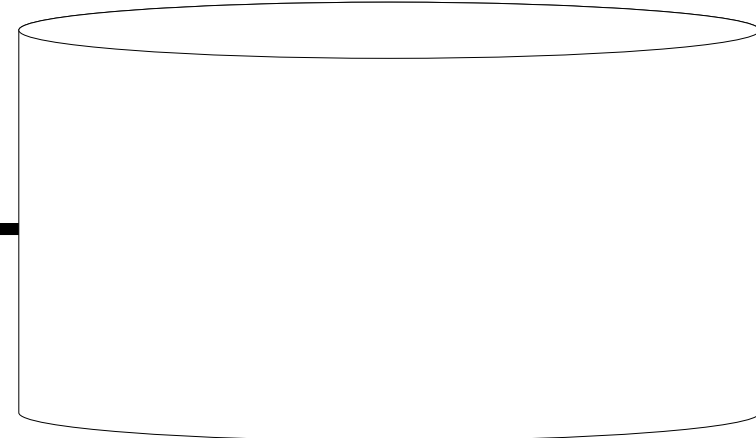
Monitor / Teclado

Digite x e y: 5 8 <ENTER>

Memória

x	y	soma
5	8	
000	001	002
prod		
003	004	005
006	007	008

HD



Execução

```
#include <stdio.h>
void operacoes(int a, int b, int*
res1, int* res2){
    *res1 = a + b;
    *res2 = a * b;
}
int main() {
    int x, y, soma, prod;
    printf("Digite x e y: ");
    scanf("%d %d", &x, &y);
    operacoes(x, y, &soma, &prod);
    printf("soma = %d, prod =
%d", soma, prod);
    return 0;
}
```

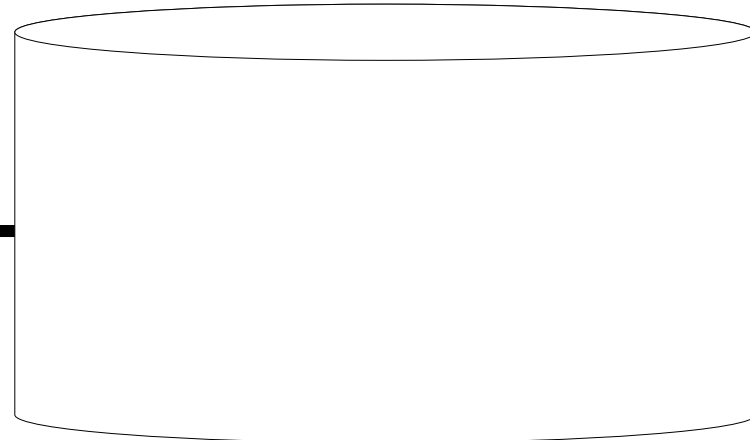
Monitor / Teclado

Digite x e y: 5 8 <ENTER>

Memória

x	y	soma
5	8	
000	001	002
prod	a _{operacoes()}	b _{operacoes()}
	5	8
003	004	005
res1 _{operacoes()}	res2 _{operacoes()}	
002	003	
006	007	008

HD



As variáveis *a*, *b*, *res1* e *res2* são criadas na memória. Estas variáveis são exclusivas da função *operacoes()* e ocupa outro espaço na memória diferente de *x*, *y*, *soma* e *prod*.

Execução

```
#include <stdio.h>
void operacoes(int a,int b,int*
res1,int* res2){
    *res1 = a + b;
    *res2 = a * b;
}
int main() {
    int x, y, soma, prod;
    printf("Digite x e y: ");
    scanf("%d %d", &x, &y);
    operacoes(x, y, &soma , &prod);
    printf("soma = %d, prod =
%d", soma, prod);
    return 0;
}
```

$a_{operacoes()} + b_{operacoes()}$
 $= 5 + 8$
 $= 13.$

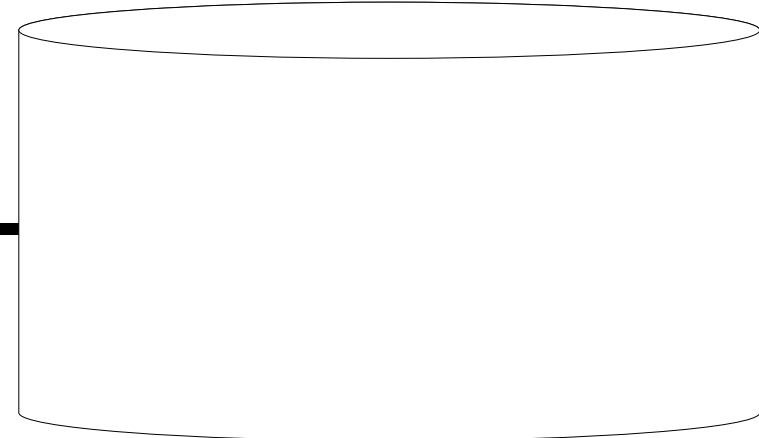
Monitor / Teclado

Digite x e y: 5 8 <ENTER>

Memória

x	y	soma
5	8	
000	001	002
prod	a _{operacoes()}	b _{operacoes()}
	5	8
003	004	005
res1 _{operacoes()}	res2 _{operacoes()}	
002	003	
006	007	008

HD



Execução

```
#include <stdio.h>
void operacoes(int a,int b,int*
res1,int* res2){
    *res1 = a + b;
    *res2 = a * b;
}
int main() {
    int x, y, soma, prod;
    printf("Digite x e y: ");
    scanf("%d %d", &x, &y);
    operacoes(x, y, &soma , &prod);
    printf("soma = %d, prod =
%d", soma, prod);
    return 0;
}
```

Monitor / Teclado

Digite x e y: 5 8 <ENTER>

Memória

x	y	soma
5	8	13
000	001	002
prod	a _{operacoes()}	b _{operacoes()}
	5	8
003	004	005
res1 _{operacoes()}	res2 _{operacoes()}	
002	003	
006	007	008

HD

1. O computador vai até **res1_{operacoes()}*
2. Pega seu conteúdo (002)
3. Vai até 002
4. Atribui 5+8 (13) neste endereço (002)

Execução

```
#include <stdio.h>
void operacoes(int a,int b,int*
res1,int* res2){
    *res1 = a + b;
    *res2 = a * b;
}
int main() {
    int x, y, soma, prod;
    printf("Digite x e y: ");
    scanf("%d %d", &x, &y);
    operacoes(x, y, &soma , &prod);
    printf("soma = %d, prod =
%d", soma, prod);
    return 0;
}
```

$a_{operacoes()} * b_{operacoes()}$
 $= 5 * 8$
 $= 40$

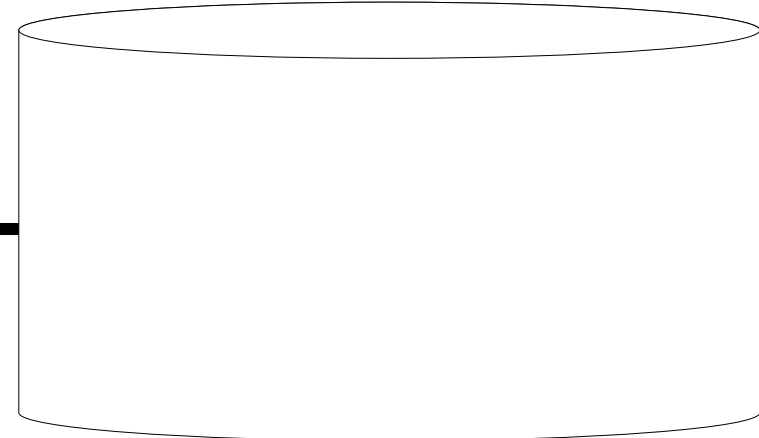
Monitor / Teclado

Digite x e y: 5 8 <ENTER>

Memória

x	y	soma
5	8	13
000	001	002
prod	a _{operacoes()}	b _{operacoes()}
	5	8
003	004	005
res1 _{operacoes()}	res2 _{operacoes()}	
002	003	
006	007	008

HD



Execução

```
#include <stdio.h>
void operacoes(int a,int b,int*
res1,int* res2){
    *res1 = a + b;
    *res2 = a * b;
}
int main() {
    int x, y, soma, prod;
    printf("Digite x e y: ");
    scanf("%d %d", &x, &y);
    operacoes(x, y, &soma , &prod);
    printf("soma = %d, prod =
%d", soma, prod);
    return 0;
}
```

Monitor / Teclado

Digite x e y: 5 8 <ENTER>

Memória

x	y	soma
5	8	13
000	001	002
prod	a _{operacoes()}	b _{operacoes()}
40	5	8
003	004	005
res1 _{operacoes()}	res2 _{operacoes()}	
002	003	
006	007	008

HD

1. O computador vai até **res2_{operacoes()}*
2. Pega seu conteúdo (003)
3. Vai até 003
4. Atribui 5*8 (40) neste endereço (003)

Execução

```
#include <stdio.h>
void operacoes(int a,int b,int*
res1,int* res2){
    *res1 = a + b;
    *res2 = a * b;
}
int main() {
    int x, y, soma, prod;
    printf("Digite x e y: ");
    scanf("%d %d", &x, &y);
    operacoes(x, y, &soma , &prod);
    printf("soma = %d, prod =
%d", soma, prod);
    return 0;
}
```

Monitor / Teclado

Digite x e y: 5 8 <ENTER>

Memória

x	y	soma
5	8	13
000	001	002
prod		
40		
003	004	005
006	007	008

HD

Como a função operacoes() não retorna nada, as variáveis são perdidas. Porém, note que o uso de ponteiros permitiu operacoes() armazenar resultados em soma e prod do main().

Execução

```
#include <stdio.h>
void operacoes(int a,int b,int*
res1,int* res2){
    *res1 = a + b;
    *res2 = a * b;
}
int main() {
    int x, y, soma, prod;
    printf("Digite x e y: ");
    scanf("%d %d", &x, &y);
    operacoes(x, y, &soma , &prod);
    printf("soma = %d, prod =
%d", soma, prod);
    return 0;
}
```

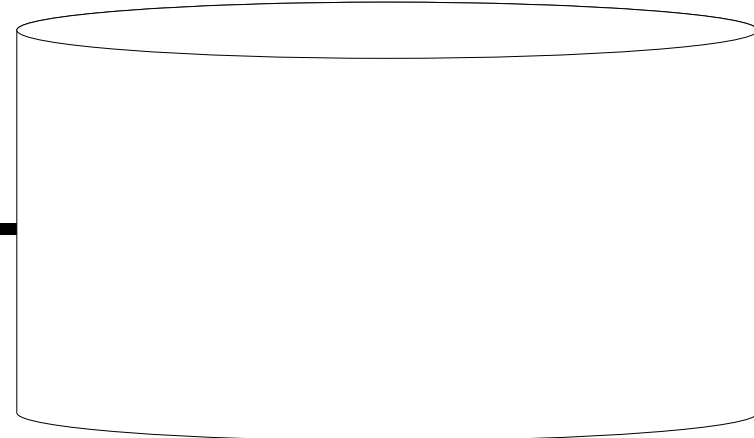
Monitor / Teclado

Digite x e y: 5 8 <ENTER>
soma = 13, prod = 40

Memória

x	y	soma
5	8	13
000	001	002
prod		
40		
003	004	005
006	007	008

HD



Execução

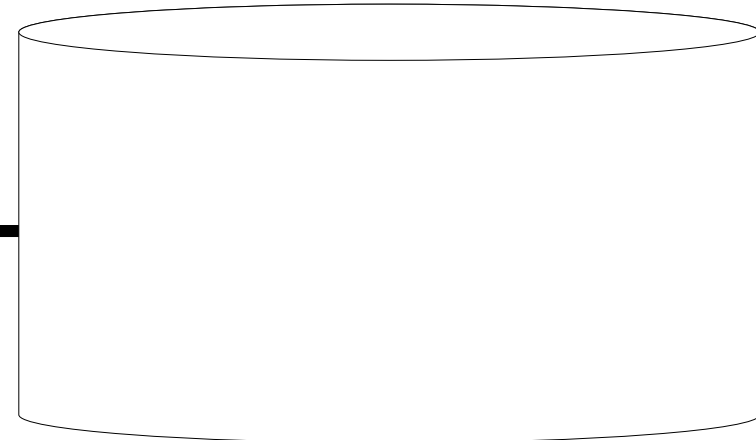
```
#include <stdio.h>
void operacoes(int a,int b,int*
res1,int* res2){
    *res1 = a + b;
    *res2 = a * b;
}
int main() {
    int x, y, soma, prod;
    printf("Digite x e y: ");
    scanf("%d %d", &x, &y);
    operacoes(x, y, &soma , &prod);
    printf("soma = %d, prod =
%d", soma, prod);
    return 0;
}
```

Monitor / Teclado

Memória

000	001	002
003	004	005
006	007	008

HD



Ponteiros

O que aparece na tela?

```
#include <stdio.h>
struct ponto {
    int x;
    int y;
};
int main() {
    struct ponto q = {1,3};
    struct ponto *p;
    p = &q;
    printf("q.x = %d\n", q.x);
    printf("q.y = %d\n", q.y);
    printf("&q = %p\n", &q);
    printf("p = %p\n", p);
    printf("&(q.x) = %p\n", &(q.x));
    printf("&(q.y) = %p\n", &(q.y));
    printf("( *p ).x = %d\n", (*p).x);
    printf("( *p ).y = %d\n", (*p).y);
    return 0;
}
```

Execução

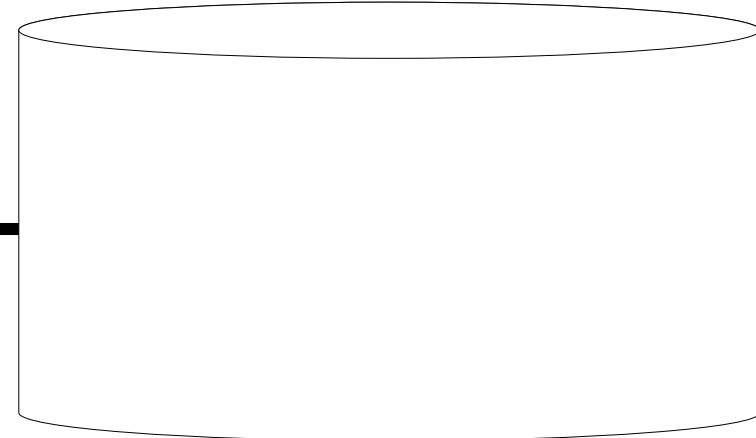
```
#include <stdio.h>
struct ponto {
    int x;
    int y;
};
int main() {
    struct ponto q = {1,3};
    struct ponto *p;
    p = &q;
    printf("q.x = %d\n", q.x);
    printf("q.y = %d\n", q.y);
    printf("&q = %p\n", &q);
    printf("p = %p\n", p);
    printf("&(q.x) = %p\n", &(q.x));
    printf("&(q.y) = %p\n", &(q.y));
    printf("( *p ).x = %d\n", (*p).x);
    printf("( *p ).y = %d\n", (*p).y);
    return 0;
}
```

Monitor / Teclado

Memória

000	001	002
003	004	005
006	007	008

HD



Execução

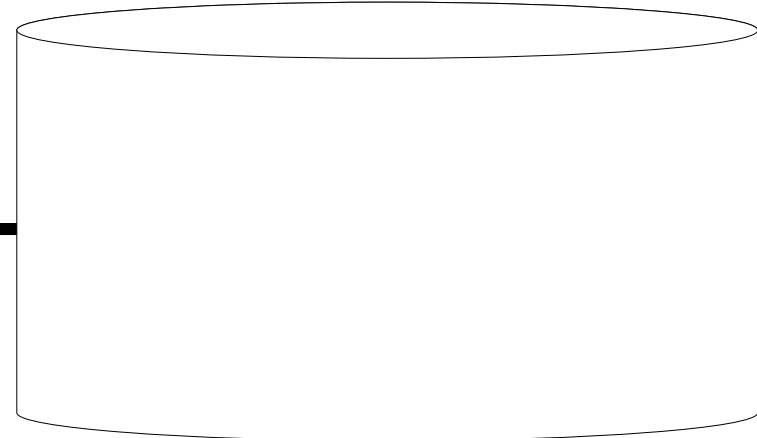
```
#include <stdio.h>
struct ponto {
    int x;
    int y;
};
int main() {
    struct ponto q = {1, 3};
    struct ponto *p;
    p = &q;
    printf("q.x = %d\n", q.x);
    printf("q.y = %d\n", q.y);
    printf("&q = %p\n", &q);
    printf("p = %p\n", p);
    printf("&(q.x) = %p\n", &(q.x));
    printf("&(q.y) = %p\n", &(q.y));
    printf("( *p ).x = %d\n", (*p).x);
    printf("( *p ).y = %d\n", (*p).y);
    return 0;
}
```

Monitor / Teclado

Memória

q			
x		y	
000	1	001	3
002			
003		004	005
006		007	008

HD



Execução

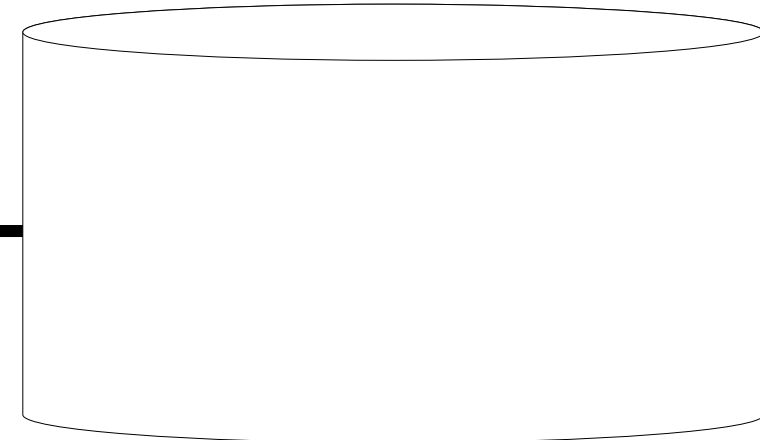
```
#include <stdio.h>
struct ponto {
    int x;
    int y;
};
int main() {
    struct ponto q = {1, 3};
    struct ponto *p;
    p = &q;
    printf("q.x = %d\n", q.x);
    printf("q.y = %d\n", q.y);
    printf("&q = %p\n", &q);
    printf("p = %p\n", p);
    printf("&(q.x) = %p\n", &(q.x));
    printf("&(q.y) = %p\n", &(q.y));
    printf("( *p ).x = %d\n", (*p).x);
    printf("( *p ).y = %d\n", (*p).y);
    return 0;
}
```

Monitor / Teclado

Memória

q		p
x	y	
000 1	001 3	002
003	004	005
006	007	008

HD



Execução

```
#include <stdio.h>
struct ponto {
    int x;
    int y;
};
int main() {
    struct ponto q = {1, 3};
    struct ponto *p;
    p = &q;
    printf("q.x = %d\n", q.x);
    printf("q.y = %d\n", q.y);
    printf("&q = %p\n", &q);
    printf("p = %p\n", p);
    printf("&(q.x) = %p\n", &(q.x));
    printf("&(q.y) = %p\n", &(q.y));
    printf("( *p ).x = %d\n", (*p).x);
    printf("( *p ).y = %d\n", (*p).y);
    return 0;
}
```

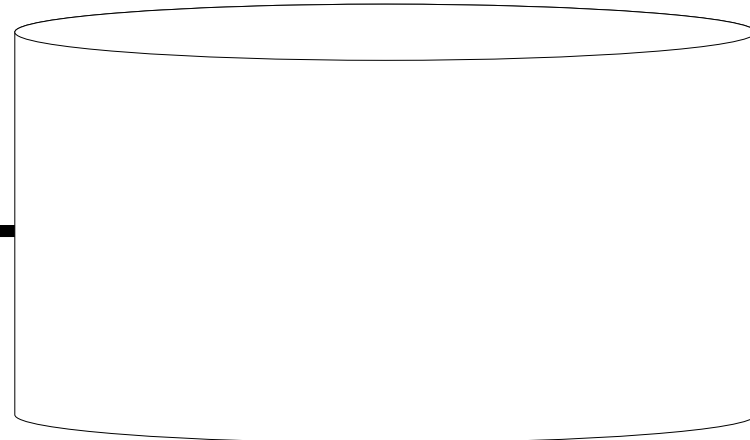
q é um struct e, portanto, um tipo de dado hierárquico. C define como o endereço de q o endereço do primeiro elemento da struct.

Monitor / Teclado

Memória

q		p	000
x	y		
000 1	001 3	002	
003	004	005	
006	007	008	

HD



Execução

```
#include <stdio.h>
struct ponto {
    int x;
    int y;
};
int main() {
    struct ponto q = {1, 3};
    struct ponto *p;
    p = &q;
    printf("q.x = %d\n", q.x);
    printf("q.y = %d\n", q.y);
    printf("&q = %p\n", &q);
    printf("p = %p\n", p);
    printf("&(q.x) = %p\n", &(q.x));
    printf("&(q.y) = %p\n", &(q.y));
    printf("( *p ).x = %d\n", (*p).x);
    printf("( *p ).y = %d\n", (*p).y);
    return 0;
}
```

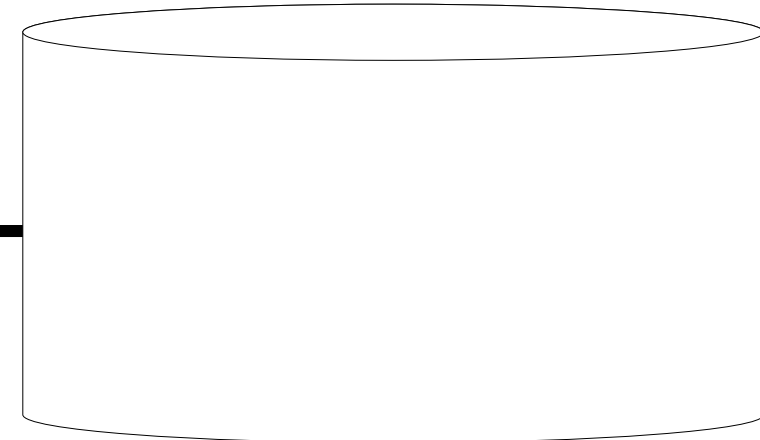
Monitor / Teclado

q.x = 1

Memória

q		p	
x	y		
000	1	001	3
002		002	
003		004	
005		005	
006		007	
008		008	

HD



Execução

```
#include <stdio.h>
struct ponto {
    int x;
    int y;
};
int main() {
    struct ponto q = {1, 3};
    struct ponto *p;
    p = &q;
    printf("q.x = %d\n", q.x);
    printf("q.y = %d\n", q.y);
    printf("&q = %p\n", &q);
    printf("p = %p\n", p);
    printf("&(q.x) = %p\n", &(q.x));
    printf("&(q.y) = %p\n", &(q.y));
    printf("( *p ).x = %d\n", (*p).x);
    printf("( *p ).y = %d\n", (*p).y);
    return 0;
}
```

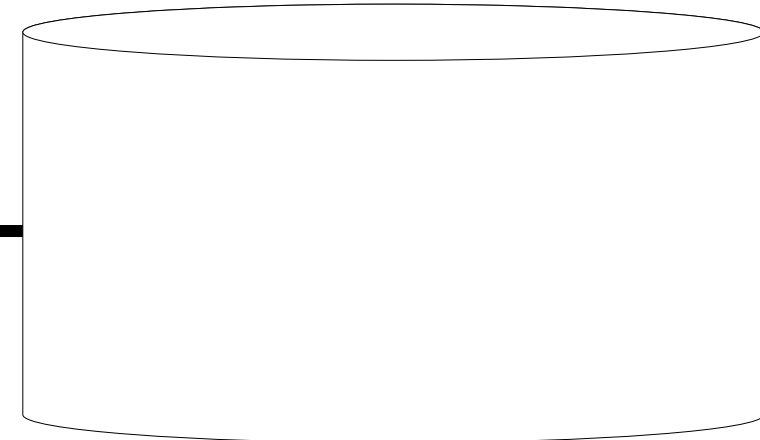
Monitor / Teclado

q.x = 1
q.y = 3

Memória

q		p	
x	y	000	
000 1	001 3	002	
003		004	005
006		007	008

HD



Execução

```
#include <stdio.h>
struct ponto {
    int x;
    int y;
};
int main() {
    struct ponto q = {1, 3};
    struct ponto *p;
    p = &q;
    printf("q.x = %d\n", q.x);
    printf("q.y = %d\n", q.y);
    printf("&q = %p\n", &q);
    printf("p = %p\n", p);
    printf("&(q.x) = %p\n", &(q.x));
    printf("&(q.y) = %p\n", &(q.y));
    printf("( *p ).x = %d\n", (*p).x);
    printf("( *p ).y = %d\n", (*p).y);
    return 0;
}
```

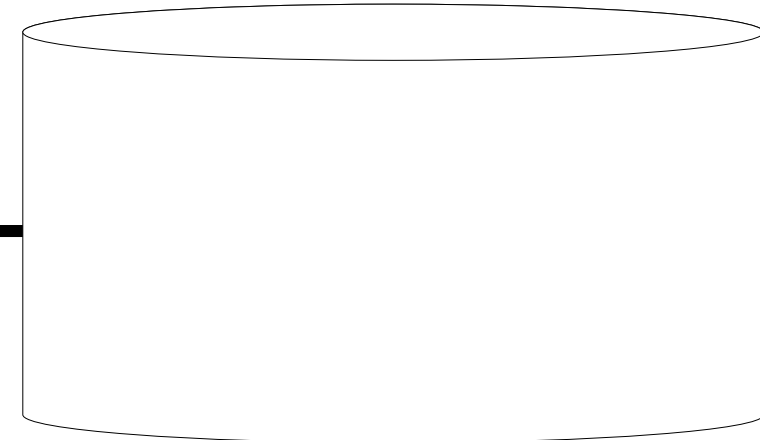
Monitor / Teclado

q.x = 1
q.y = 3

Memória

q		p	
x	y	000	
000 1	001 3	002	
003		004	005
006		007	008

HD



Execução

```
#include <stdio.h>
struct ponto {
    int x;
    int y;
};
int main() {
    struct ponto q = {1, 3};
    struct ponto *p;
    p = &q;
    printf("q.x = %d\n", q.x);
    printf("q.y = %d\n", q.y);
    printf("&q = %p\n", &q);
    printf("p = %p\n", p);
    printf("&(q.x) = %p\n", &(q.x));
    printf("&(q.y) = %p\n", &(q.y));
    printf("( *p ).x = %d\n", (*p).x);
    printf("( *p ).y = %d\n", (*p).y);
    return 0;
}
```

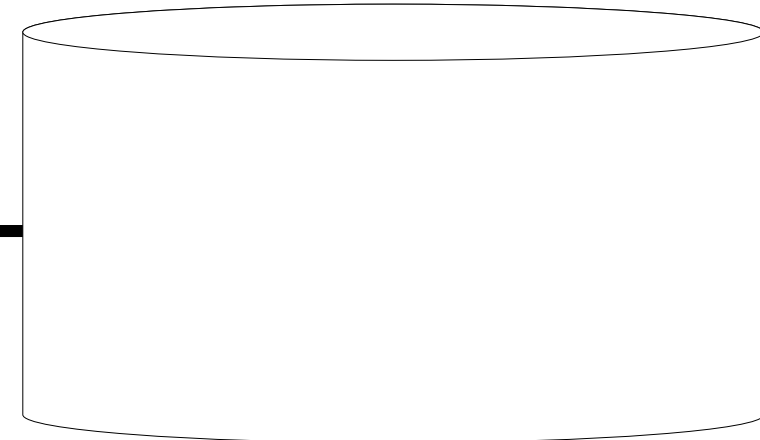
Monitor / Teclado

q.x = 1
q.y = 3
&q = 000

Memória

q		p	
x	y	000	
000 1	001 3	002	
003	004	005	
006	007	008	

HD



Execução

```
#include <stdio.h>
struct ponto {
    int x;
    int y;
};
int main() {
    struct ponto q = {1, 3};
    struct ponto *p;
    p = &q;
    printf("q.x = %d\n", q.x);
    printf("q.y = %d\n", q.y);
    printf("&q = %p\n", &q);
    printf("p = %p\n", p);
    printf("&(q.x) = %p\n", &(q.x));
    printf("&(q.y) = %p\n", &(q.y));
    printf("( *p ).x = %d\n", (*p).x);
    printf("( *p ).y = %d\n", (*p).y);
    return 0;
}
```

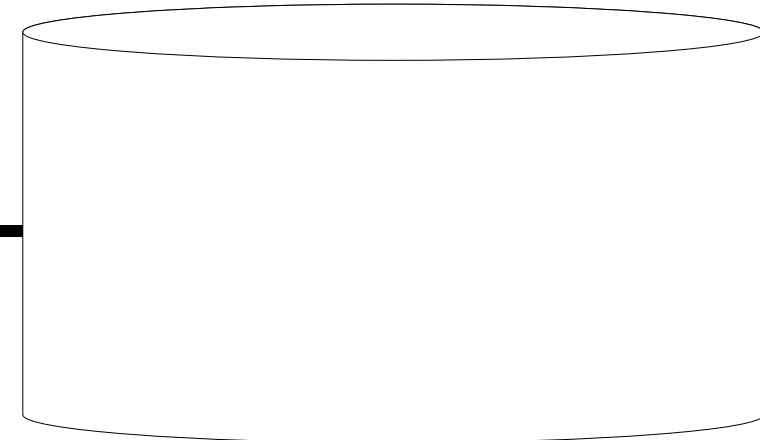
Monitor / Teclado

q.x = 1
q.y = 3
&q = 000
p = 000

Memória

q		p	
x	y	000	
000 1	001 3	002	
003	004	005	
006	007	008	

HD



Execução

```
#include <stdio.h>
struct ponto {
    int x;
    int y;
};
int main() {
    struct ponto q = {1, 3};
    struct ponto *p;
    p = &q;
    printf("q.x = %d\n", q.x);
    printf("q.y = %d\n", q.y);
    printf("&q = %p\n", &q);
    printf("p = %p\n", p);
    printf("&(q.x) = %p\n", &(q.x));
    printf("&(q.y) = %p\n", &(q.y));
    printf("( *p ).x = %d\n", (*p).x);
    printf("( *p ).y = %d\n", (*p).y);
    return 0;
}
```

1. O computador vai em q
2. Depois, em x
3. E depois pega o endereço de q.x

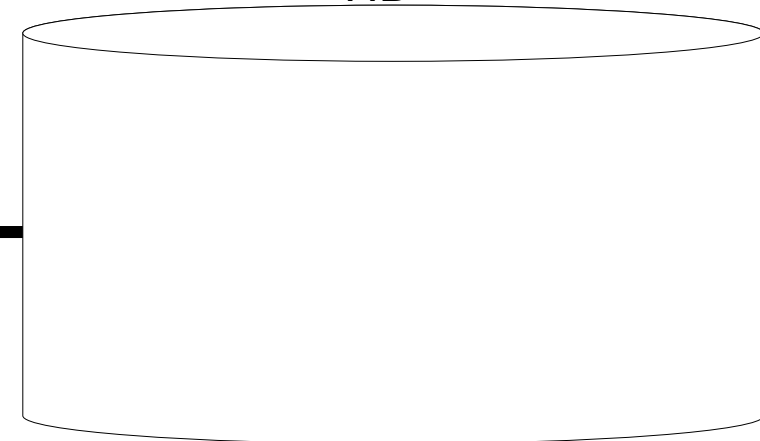
Monitor / Teclado

q.y = 3
&q = 000
p = 000
&(q.x) = 000

Memória

q		p	
x	y	000	
000 1	001 3	002	
003	004	005	
006	007	008	

HD



Execução

```
#include <stdio.h>
struct ponto {
    int x;
    int y;
};
int main() {
    struct ponto q = {1, 3};
    struct ponto *p;
    p = &q;
    printf("q.x = %d\n", q.x);
    printf("q.y = %d\n", q.y);
    printf("&q = %p\n", &q);
    printf("p = %p\n", p);
    printf("&(q.x) = %p\n", &(q.x));
    printf("&(q.y) = %p\n", &(q.y));
    printf("( *p ).x = %d\n", (*p).x);
    printf("( *p ).y = %d\n", (*p).y);
    return 0;
}
```

1. O computador vai em q
2. Depois, em y
3. E depois pega o endereço de q.y

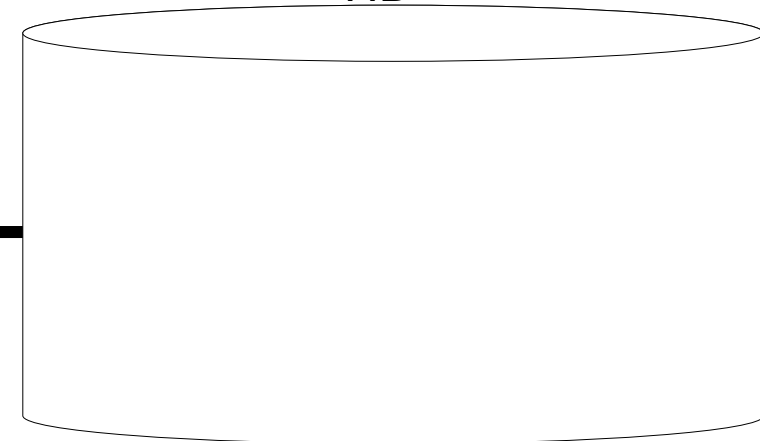
Monitor / Teclado

&q = 000
p = 000
&(q.x) = 000
&(q.y) = 001

Memória

q		p	
x	y	000	
000 1	001 3	002	
003	004	005	
006	007	008	

HD



Execução

```
#include <stdio.h>
struct ponto {
    int x;
    int y;
};
int main() {
    struct ponto q = {1, 3};
    struct ponto *p;
    p = &q;
    printf("q.x = %d\n", q.x);
    printf("q.y = %d\n", q.y);
    printf("&q = %p\n", &q);
    printf("p = %p\n", p);
    printf("&(q.x) = %p\n", &(q.x));
    printf("&(q.y) = %p\n", &(q.y));
    printf("( *p ).x = %d\n", (*p).x);
    printf("( *p ).y = %d\n", (*p).y);
    return 0;
}
```

1. O computador vai em p
2. Pega o endereço que ele armazena (000)
3. Vai até o endereço 000

4. E acessa o valor do campo x

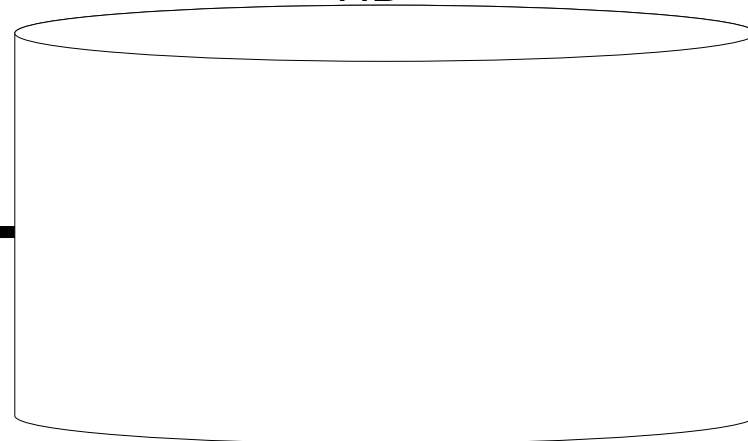
Monitor / Teclado

p = 000
&(q.x) = 000
&(q.y) = 001
(*p).x = 1

Memória

q		p	
x	y		
000	1	001	3
003		004	
006		007	

HD



Execução

```
#include <stdio.h>
struct ponto {
    int x;
    int y;
};
int main() {
    struct ponto q = {1, 3};
    struct ponto *p;
    p = &q;
    printf("q.x = %d\n", q.x);
    printf("q.y = %d\n", q.y);
    printf("&q = %p\n", &q);
    printf("p = %p\n", p);
    printf("&(q.x) = %p\n", &(q.x));
    printf("&(q.y) = %p\n", &(q.y));
    printf("( *p ). x = %d\n", ( *p ). x);
    printf("( *p ). y = %d\n", ( *p ). y);
    return 0;
}
```

1. O computador vai em p
2. Pega o endereço que ele armazena (000)
3. Vai até o endereço 000
4. E acessa o valor do campo y

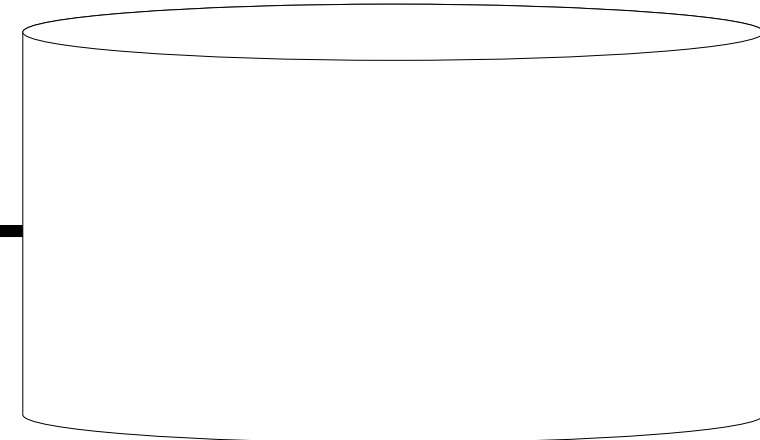
Monitor / Teclado

&(q.x) = 000
&(q.y) = 001
(*p).x = 1
(*p).y = 3

Memória

q		p	
x	y		
000	1	001	3
002			
003		004	
005			
006		007	
008			

HD



Execução

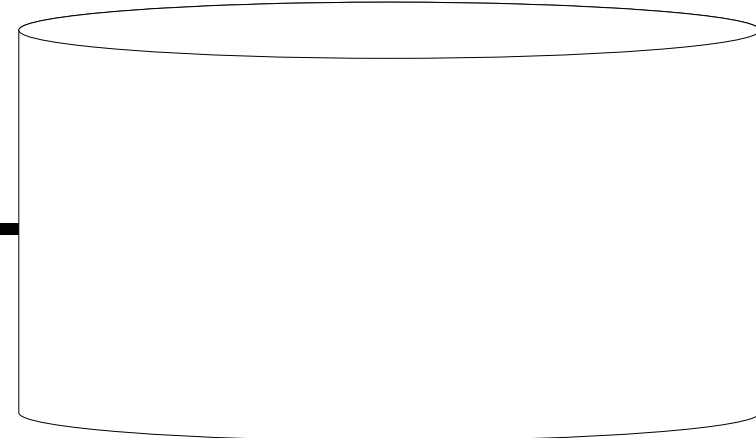
```
#include <stdio.h>
struct ponto {
    int x;
    int y;
};
int main() {
    struct ponto q = {1,3};
    struct ponto *p;
    p = &q;
    printf("q.x = %d\n", q.x);
    printf("q.y = %d\n", q.y);
    printf("&q = %p\n", &q);
    printf("p = %p\n", p);
    printf("&(q.x) = %p\n", &(q.x));
    printf("&(q.y) = %p\n", &(q.y));
    printf("( *p ).x = %d\n", (*p).x);
    printf("( *p ).y = %d\n", (*p).y);
    return 0;
}
```

Monitor / Teclado

Memória

000	001	002
003	004	005
006	007	008

HD



Ponteiros

*Optional: ao invés de usar (*p).x e (*p).y
pode-se usar p->x e p->y.*

```
#include <stdio.h>
struct ponto {
    int x;
    int y;
};
int main() {
    struct ponto q = {1,3};
    struct ponto *p;
    p = &q;
    printf("q.x = %d\n", q.x);
    printf("q.y = %d\n", q.y);
    printf("&q = %p\n", &q);
    printf("p = %p\n", p);
    printf("&(q.x) = %p\n", &(q.x));
    printf("&(q.y) = %p\n", &(q.y));

    printf("( *p ).x = %d\n", p->x);

    printf("( *p ).y = %d\n", p->y);
    return 0;
}
```

Ponteiros

Computação Eletrônica

Juliano M. Iyoda
jmi@cin.ufpe.br